

アルゴリズムとデータ構造

今日学ぶこと

- データ構造の作り方
- コレクションとデータの表現
 - データの貯め方: Dynamic Array, Linked List
 - 点と枝の表現: グラフと木
 - 順番に整理: Stack, Queue, Deque, Priority Queue (& Heap)
 - 大小で整理: Search Tree
 - 同じかどうかで整理: Hash Table, HashMap, Set
- 基礎的なアルゴリズム
 - 探索アルゴリズム
 - Sort Algorithm
 - Graph Algorithm
 - Matching Algorithm

データの要素

全てはビット列から作られる

- 1bit: 0 か 1
- n個のbitを並べたもの: n-bit
- 整数 (integer) -- 8bit, 16bit, 32bit, 64bit
 - 文字
 - 'A': 0x41 in ASCII (0100_0001)
 - 'あ': 0x3042 in Unicode
 - 機械語
 - 0x90: NOP in x86
 - 0x4889e5: mov rax, rdi – rdi の値を rax にコピー
 - メモリアドレス (64bit)
 - データがメモリのどこにあるかを示す数値
- 浮動小数 (floating point number, 32bit, 64bit, 128bit)

これらを、

- 指し示したり (ID, メモリアドレス)
- 並べたり (Array: [1,2,3], ['H','e','l','l','o'])
- 組み合わせたり (Struct)

配列

データを並べたもの

data[8]

data[0]	data[1]	data[2]	data[3]	data[4]	data[5]	data[6]	data[7]
---------	---------	---------	---------	---------	---------	---------	---------

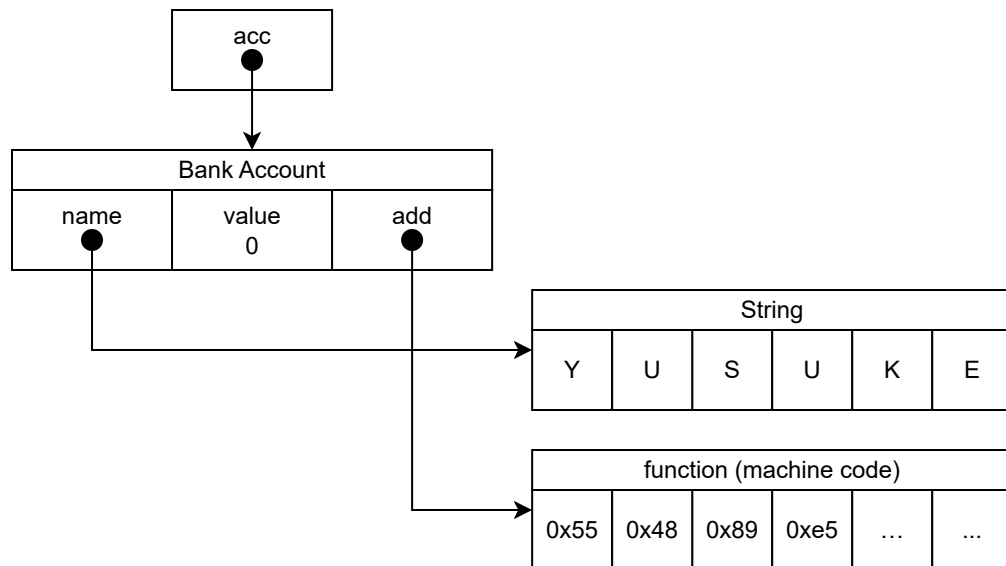
data[2][4]

data[0][0]	data[0][1]	data[0][2]	data[0][3]	data[1][0]	data[1][1]	data[1][2]	data[1][3]
------------	------------	------------	------------	------------	------------	------------	------------

構造体

データを組み合わせる

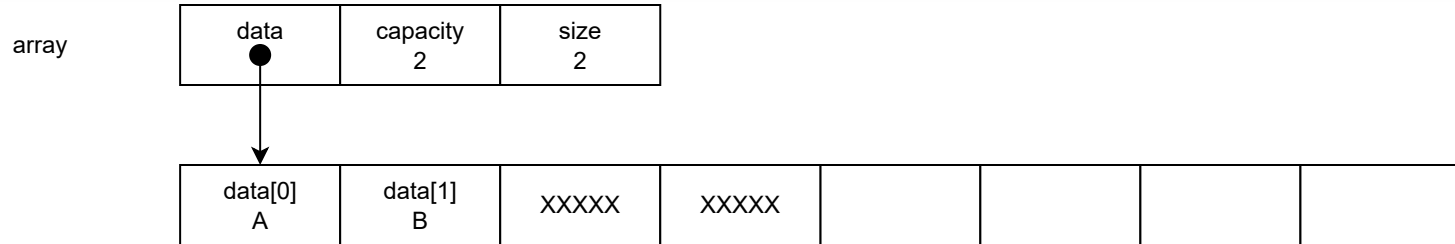
```
class BankAccount {  
    String name;  
    int value;  
    void add(int delta);  
}  
acc = new BankAccount();
```



COLLECTION - データの貯め方

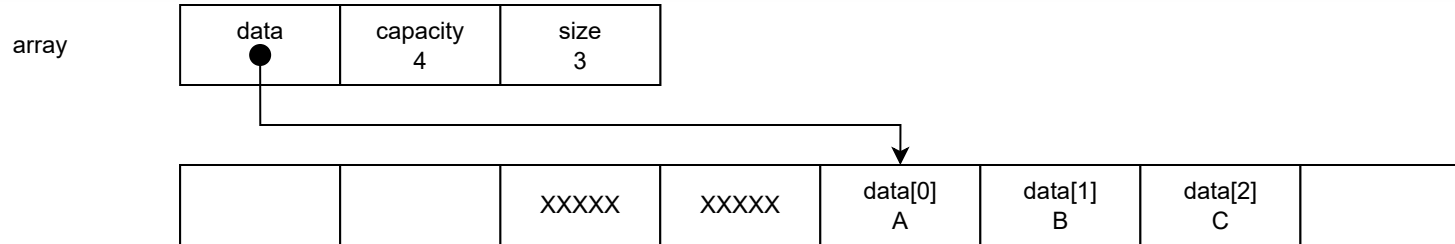
動的配列 (DYNAMIC ARRAY)

```
class DynamicArray {  
    Array data = new Array(2);  
    int capacity = 2;  
    int size = 0;  
}
```



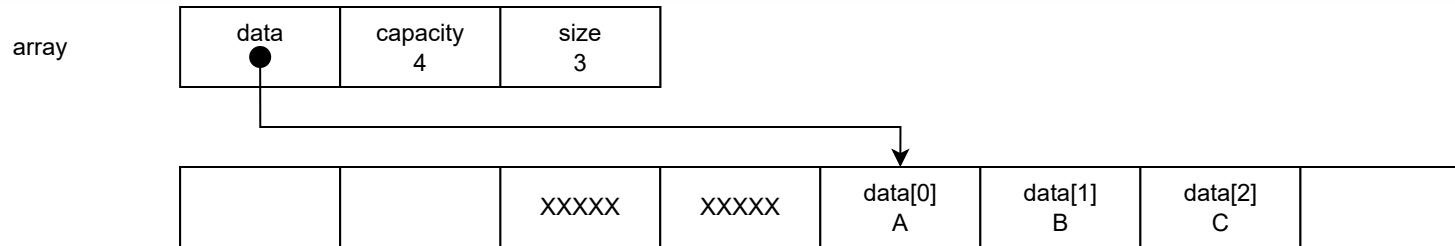
動的配列 (DYNAMIC ARRAY)

```
class DynamicArray {  
    Array data = new Array(2);  
    int capacity = 2;  
    int size = 0;  
}
```



動的配列 (DYNAMIC ARRAY)

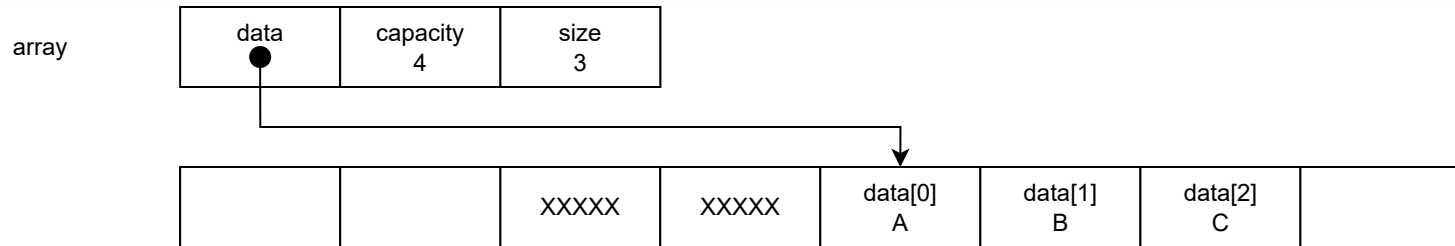
```
class DynamicArray {
    Array data = new Array(2);
    int capacity = 2;
    int size = 0;
}
```



お引越しコストは、 2^x 回のデータ追加で、 $1 + 2 + \dots + 2^x \approx 2^{x+1}$ 個分のコピー
 → 償却 $O(1)$ (定数) 時間

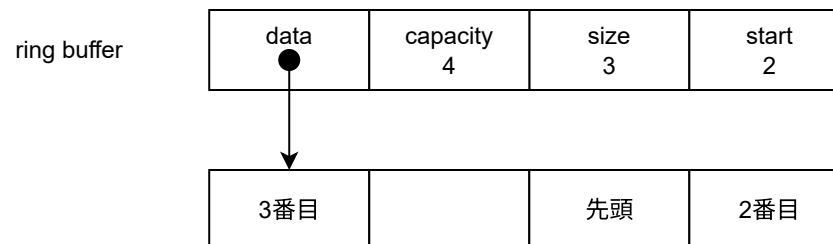
動的配列 (DYNAMIC ARRAY)

```
class DynamicArray {
    Array data = new Array(2);
    int capacity = 2;
    int size = 0;
}
```



お引越しコストは、 2^x 回のデータ追加で、 $1 + 2 + \dots + 2^x \approx 2^{x+1}$ 個分のコピー
 → 償却 $O(1)$ (定数) 時間

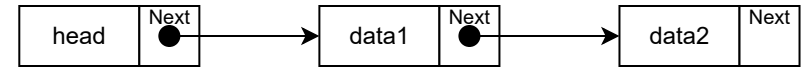
亜種: リングバッファ



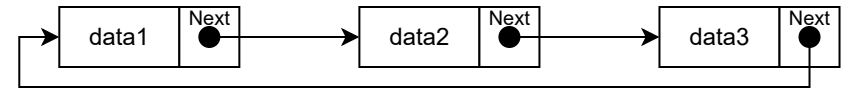
リンクリスト (LINKED LIST)

```
class LinkedList {
    int size = 0;
    LinkedListNode head = null;
    LinkedListNode tail = null;
}
class LinkedListNode {
    int data;
    LinkedListNode prev;
    LinkedListNode next;
}
```

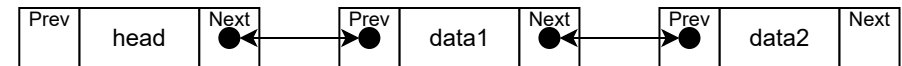
単方向 Linked List



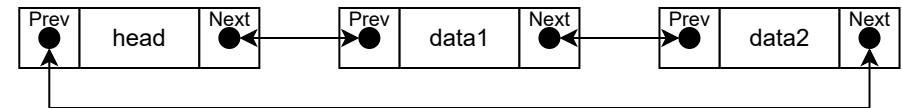
単方向 Linked List (循環)



双方向 Linked List



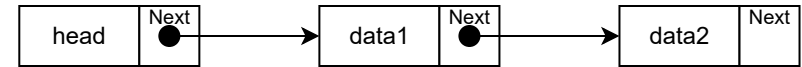
双方向 Linked List (循環)



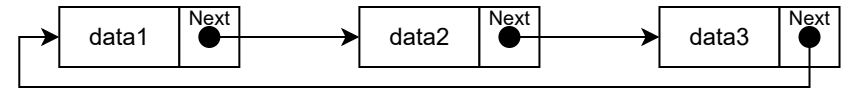
リンクリスト (LINKED LIST)

```
class LinkedList {
    int size = 0;
    LinkedListNode head = null;
    LinkedListNode tail = null;
}
class LinkedListNode {
    int data;
    LinkedListNode prev;
    LinkedListNode next;
}
```

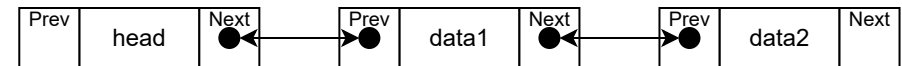
単方向 Linked List



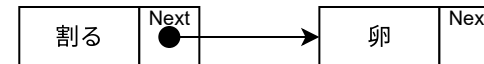
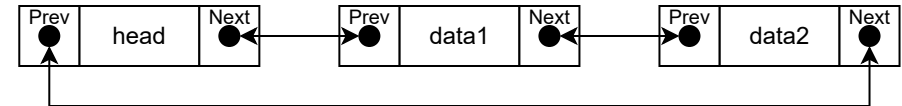
単方向 Linked List (循環)



双方向 Linked List



双方向 Linked List (循環)



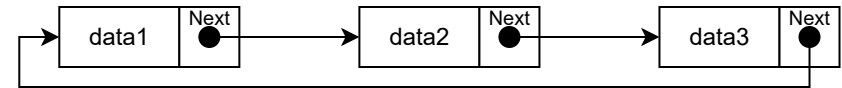
リンクリスト (LINKED LIST)

```
class LinkedList {
    int size = 0;
    LinkedListNode head = null;
    LinkedListNode tail = null;
}
class LinkedListNode {
    int data;
    LinkedListNode prev;
    LinkedListNode next;
}
```

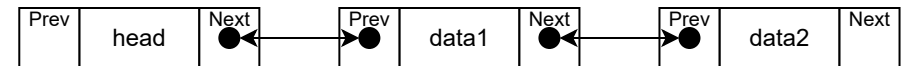
単方向 Linked List



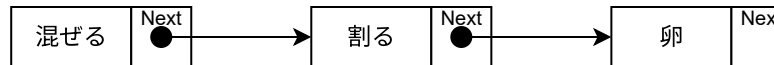
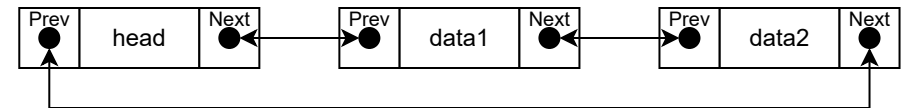
単方向 Linked List (循環)



双方向 Linked List



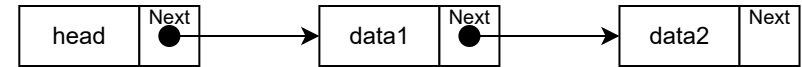
双方向 Linked List (循環)



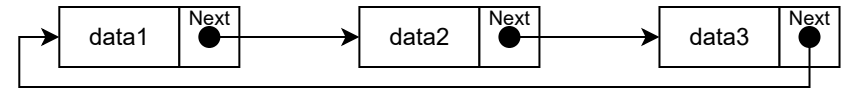
リンクリスト (LINKED LIST)

```
class LinkedList {
  int size = 0;
  LinkedListNode head = null;
  LinkedListNode tail = null;
}
class LinkedListNode {
  int data;
  LinkedListNode prev;
  LinkedListNode next;
}
```

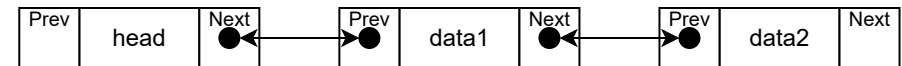
単方向 Linked List



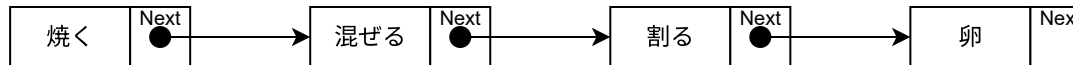
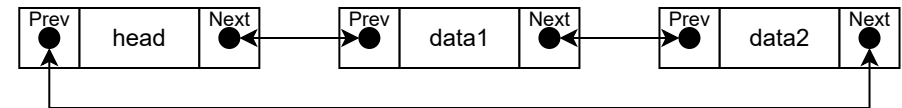
単方向 Linked List (循環)



双方向 Linked List



双方向 Linked List (循環)



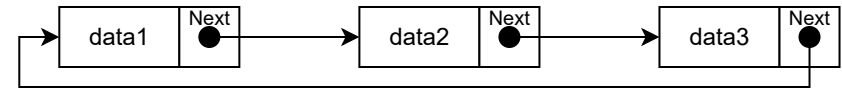
リンクリスト (LINKED LIST)

```
class LinkedList {
  int size = 0;
  LinkedListNode head = null;
  LinkedListNode tail = null;
}
class LinkedListNode {
  int data;
  LinkedListNode prev;
  LinkedListNode next;
}
```

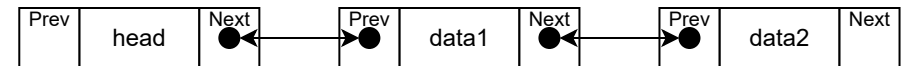
単方向 Linked List



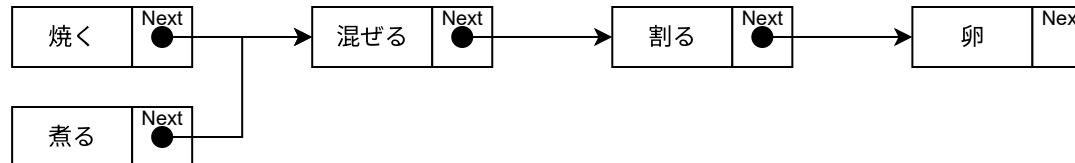
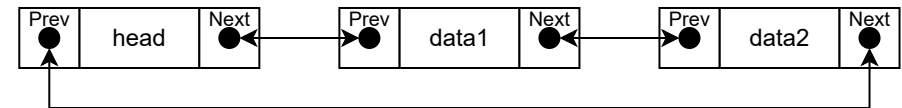
単方向 Linked List (循環)



双方向 Linked List



双方向 Linked List (循環)



動的配列 vs リンクリスト

	動的配列	リンクリスト
先頭に追加/削除	$O(N)$ / ring:償却 $O(1)$	$O(1)$
最後に追加/削除	償却 $O(1)$	$O(1)$
中央に追加/削除	$O(N)$	$O(1)$
データの探索	$O(N)$	$O(N)$
N番目にアクセス	$O(1)$	$O(N)$

動的配列は、平均して早いですが、時々すごく遅い(お引越し)

リンクリストは平均して少し遅い(毎回メモリ割当)が、安定した性能

COLLECTION - 点と枝の表現

グラフ

頂点 (vertex) と辺 (edge) の集まり

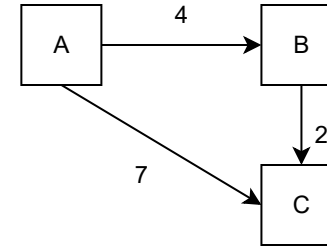
枝の行列表現

$$E = \begin{pmatrix} 0 & 4 & 7 \\ 0 & 0 & 2 \\ 0 & 0 & 0 \end{pmatrix}$$

枝の隣接リスト表現

$$E = \left\{ \begin{array}{l} A : [(B, 4), (C, 7)], \\ B : [(C, 2)] \end{array} \right\}$$

速度は行列表現が有利だが、
疎_(sparse)だと隣接リストがメモリ効率的に有利



有向_(directed) / **無向**_(undirected)

辺に向きがあるか無いか

重み付き_(weighted) / **重み無し**_(unweighted)

辺に重みをつけるか?

(無ければ全部1扱い)

連結_(connected) / **非連結**_(unconnected)

全ての点が繋がっているか?

巡回_(cyclic) / **非巡回**_(acyclic)

同じ枝を最大1回使って戻るループが存在するか?

木

木はグラフの1種。
連結で、枝の数が辺の数-1 (非巡回)

根付き木 (ROOTED TREE)

更に有向で、根を一つだけ持つもの

親・子

枝の入口の点を親、
出口の点を子と呼ぶ

根

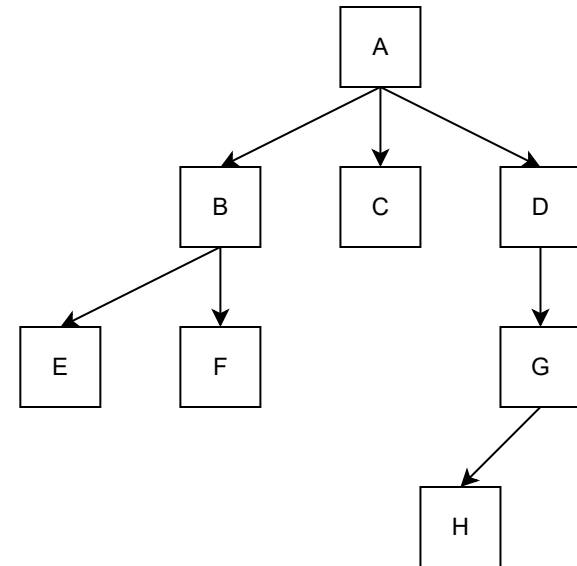
親の無い点

葉

子の無い点

N分木 (n-ary tree)

どの点の子の数もN以下である木



COLLECTION - 順番で整理

スタック / キュー / デック

1つずつ入れたり (push) 出したり (pop) する為のデータ構造

スタック / キュー / デック

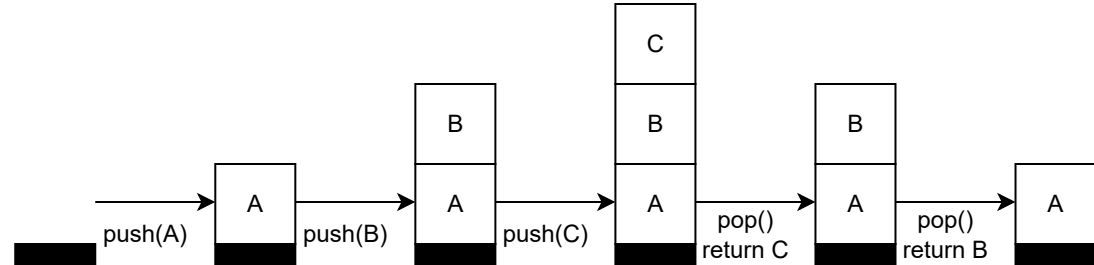
1つつ入れたり (push) 出したり (pop) する為のデータ構造

スタック (STACK)

Last In First Out

最後に入れたのが最初に出る

Stack
(LIFO)



スタック / キュー / デック

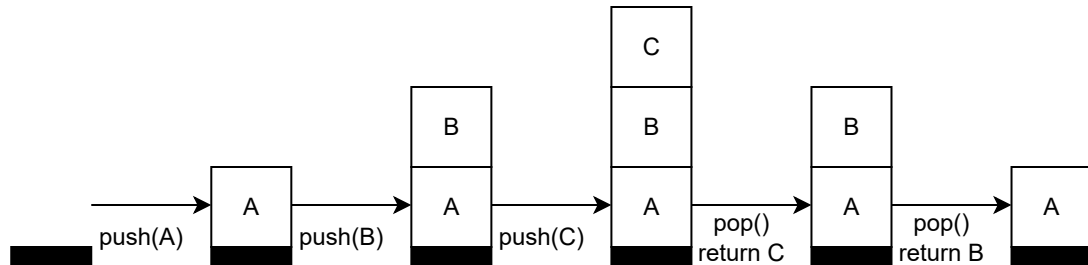
1つつ入れたり (push) 出したり (pop) する為のデータ構造

スタック (STACK)

Last In First Out

最後に入れたのが最初に出る

Stack
(LIFO)

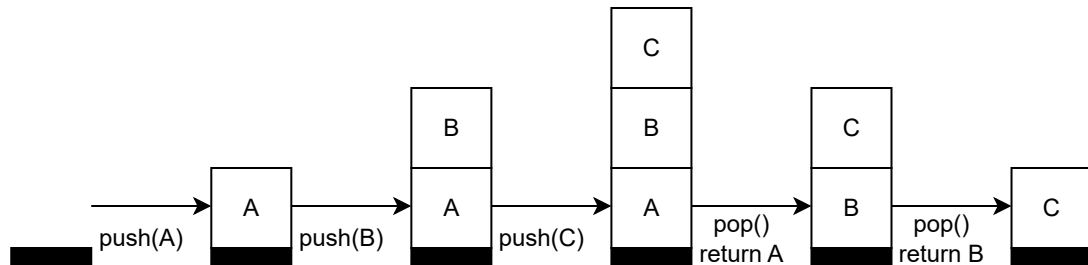


キュー (QUEUE / 待ち行列)

First In First Out

最初に入れたのが最初に出る

Queue
(FIFO)



スタック / キュー / デック

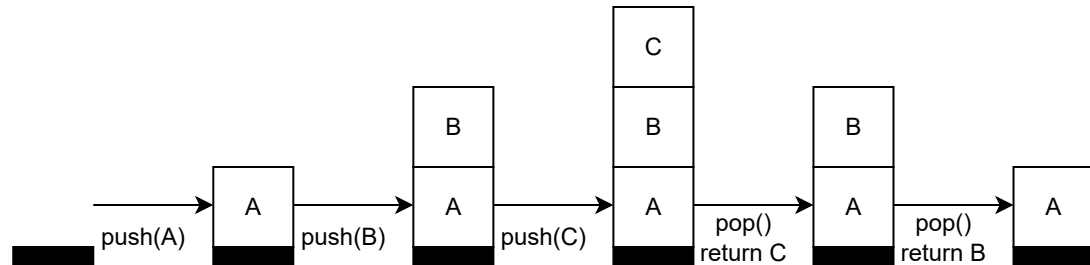
1つつ入れたり (push) 出したり (pop) する為のデータ構造

スタック (STACK)

Last In First Out

最後に入れたのが最初に出る

Stack
(LIFO)

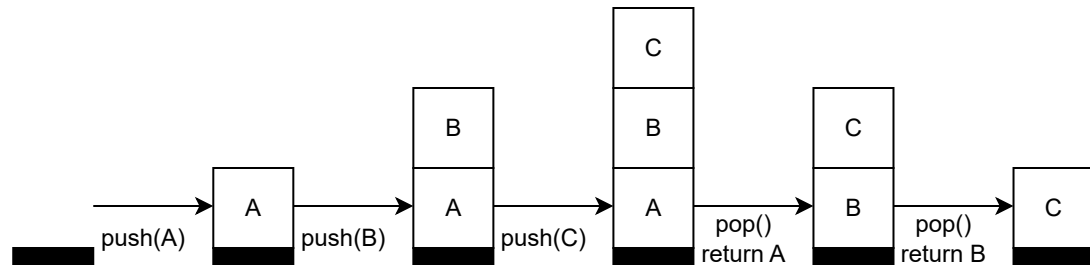


キュー (QUEUE / 待ち行列)

First In First Out

最初に入れたのが最初に出る

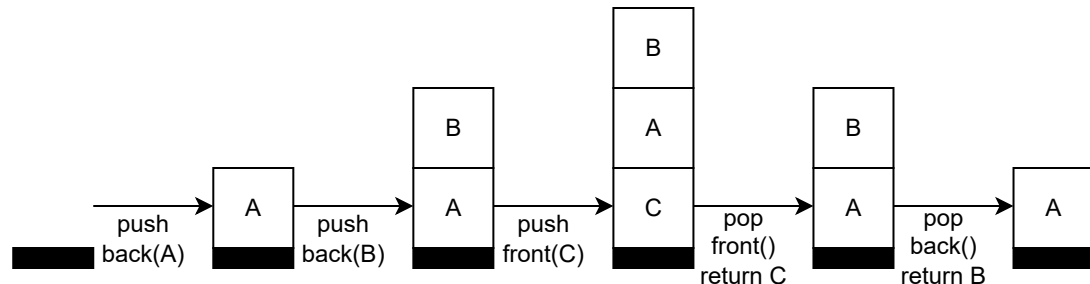
Queue
(FIFO)



デック (DEQUE / 両端キュー)

両端から入れたり出したり

Deque



スタック / キュー / デック

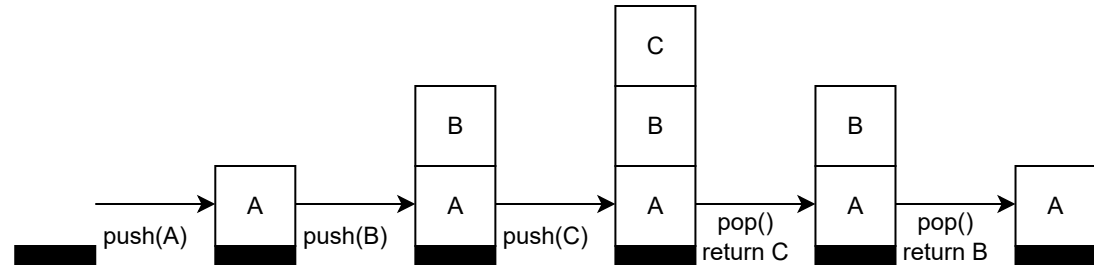
1つつ入れたり (push) 出したり (pop) する為のデータ構造

スタック (STACK)

Last In First Out

最後に入れたのが最初に出る

Stack
(LIFO)

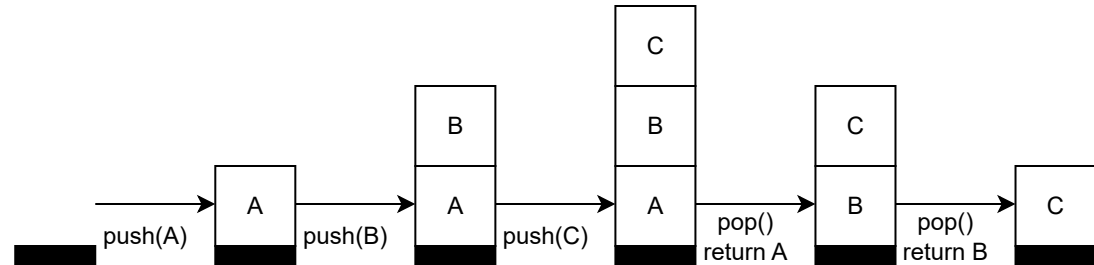


キュー (QUEUE / 待ち行列)

First In First Out

最初に入れたのが最初に出る

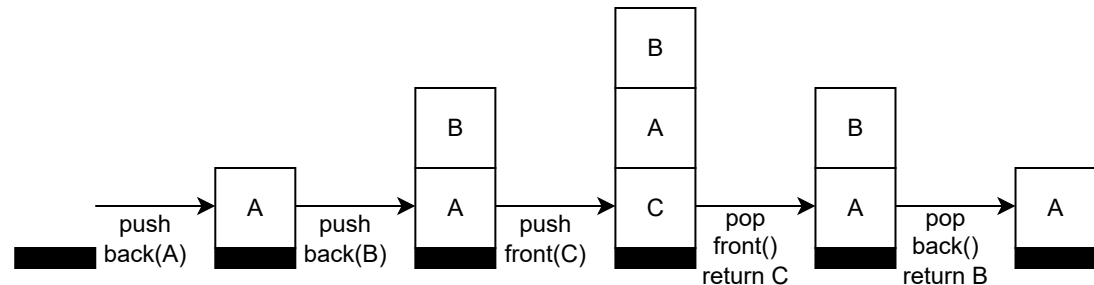
Queue
(FIFO)



デック (DEQUE / 両端キュー)

両端から入れたり出したり

Deque



動的配列 / リンクリストで実装する → push/pop が $O(1)$

ヒープ / 優先度付きキュー

優先度付きキュー (PRIORITY QUEUE)

push: 何でも入れれる。

pop: 一番小さいのが出てくる

ヒープを使って実装する

ヒープ

根付き木、かつ、二分木

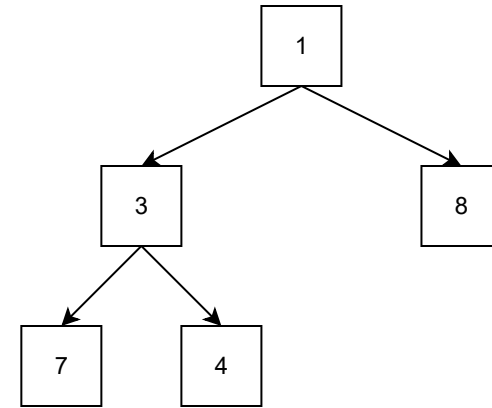
上の段から順番に、左から順番に埋まる。

親の値は子の値以下

ヒープの高さはおよそ $\log(n)$ となるので、

push / pop が $O(\log(n))$ で実装できる

データを全部ヒープに入れて、
小さい方から全部取り出すと、
 $O(n\log(n))$ でソートができる。



ヒープ / 優先度付きキュー

優先度付きキュー (PRIORITY QUEUE)

push: 何でも入れれる。

pop: 一番小さいのが出てくる

ヒープを使って実装する

ヒープ

根付き木、かつ、二分木

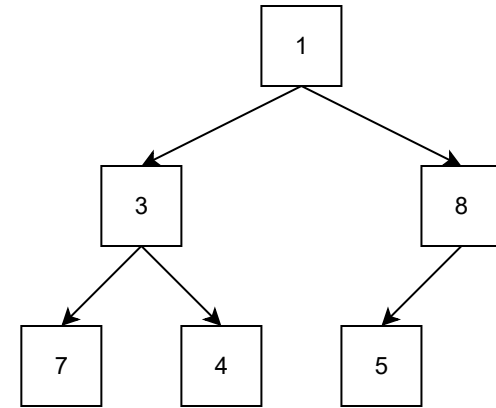
上の段から順番に、左から順番に埋まる。

親の値は子の値以下

ヒープの高さはおおよそ $\log(n)$ となるので、

push / pop が $O(\log(n))$ で実装できる

データを全部ヒープに入れて、
小さい方から全部取り出すと、
 $O(n\log(n))$ でソートができる。



5をpush。5は8より小さい。

ヒープ / 優先度付きキュー

優先度付きキュー (PRIORITY QUEUE)

push: 何でも入れれる。

pop: 一番小さいのが出てくる

ヒープを使って実装する

ヒープ

根付き木、かつ、二分木

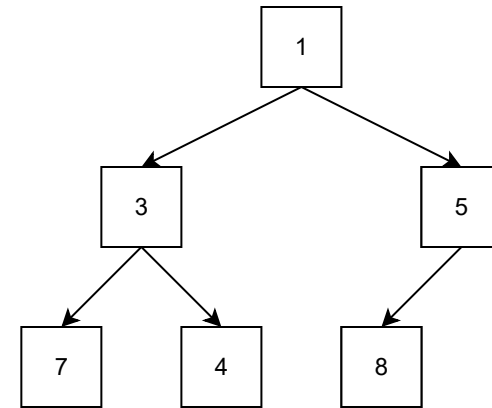
上の段から順番に、左から順番に埋まる。

親の値は子の値以下

ヒープの高さはおおよそ $\log(n)$ となるので、

push / pop が $O(\log(n))$ で実装できる

データを全部ヒープに入れて、
小さい方から全部取り出すと、
 $O(n\log(n))$ でソートができる。



5をpush。5は8より小さい。

5と8を交換。

ヒープ / 優先度付きキュー

優先度付きキュー (PRIORITY QUEUE)

push: 何でも入れれる。

pop: 一番小さいのが出てくる

ヒープを使って実装する

ヒープ

根付き木、かつ、二分木

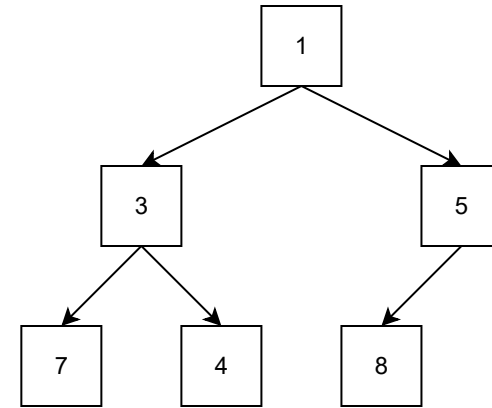
上の段から順番に、左から順番に埋まる。

親の値は子の値以下

ヒープの高さはおおよそ $\log(n)$ となるので、

push / pop が $O(\log(n))$ で実装できる

データを全部ヒープに入れて、
小さい方から全部取り出すと、
 $O(n\log(n))$ でソートができる。



ヒープ / 優先度付きキュー

優先度付きキュー (PRIORITY QUEUE)

push: 何でも入れれる。

pop: 一番小さいのが出てくる

ヒープを使って実装する

ヒープ

根付き木、かつ、二分木

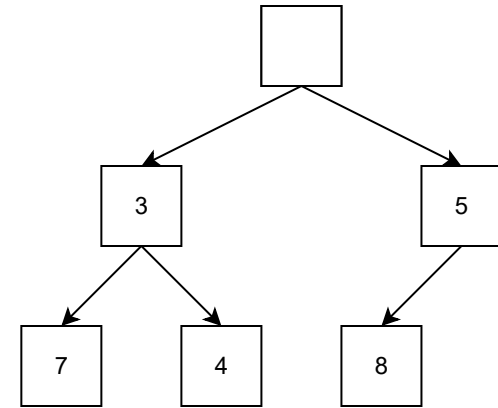
上の段から順番に、左から順番に埋まる。

親の値は子の値以下

ヒープの高さはおよそ $\log(n)$ となるので、

push / pop が $O(\log(n))$ で実装できる

データを全部ヒープに入れて、
小さい方から全部取り出すと、
 $O(n\log(n))$ でソートができる。



1を pop。

上から順に詰まるルールを満たしたい。

ヒープ / 優先度付きキュー

優先度付きキュー (PRIORITY QUEUE)

push: 何でも入れれる。

pop: 一番小さいのが出てくる

ヒープを使って実装する

ヒープ

根付き木、かつ、二分木

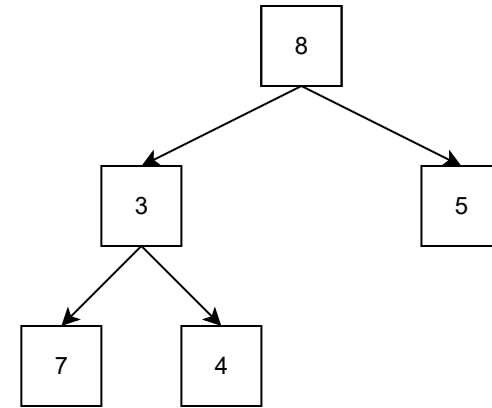
上の段から順番に、左から順番に埋まる。

親の値は子の値以下

ヒープの高さはおよそ $\log(n)$ となるので、

push / pop が $O(\log(n))$ で実装できる

データを全部ヒープに入れて、
小さい方から全部取り出すと、
 $O(n\log(n))$ でソートができる。



1を pop。

上から順に詰まるルールを満たしたい。

一番右下の8を削除し、一番上に。

8は3,5より大きい。

ヒープ / 優先度付きキュー

優先度付きキュー (PRIORITY QUEUE)

push: 何でも入れれる。

pop: 一番小さいのが出てくる

ヒープを使って実装する

ヒープ

根付き木、かつ、二分木

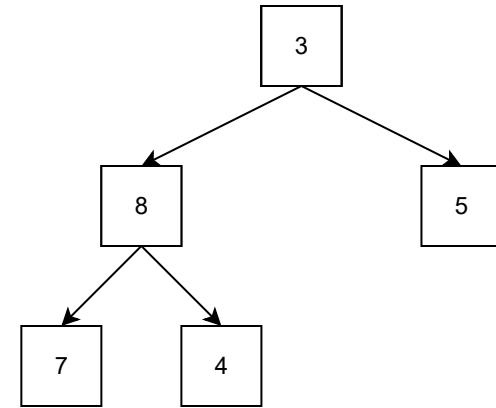
上の段から順番に、左から順番に埋まる。

親の値は子の値以下

ヒープの高さはおよそ $\log(n)$ となるので、

push / pop が $O(\log(n))$ で実装できる

データを全部ヒープに入れて、
小さい方から全部取り出すと、
 $O(n\log(n))$ でソートができる。



1を pop。

上から順に詰まるルールを満たしたい。

一番右下の8を削除し、一番上に。

8は3,5より大きい。

8と小さい3を交換。

8は4,7より小さい。

ヒープ / 優先度付きキュー

優先度付きキュー (PRIORITY QUEUE)

push: 何でも入れれる。

pop: 一番小さいのが出てくる

ヒープを使って実装する

ヒープ

根付き木、かつ、二分木

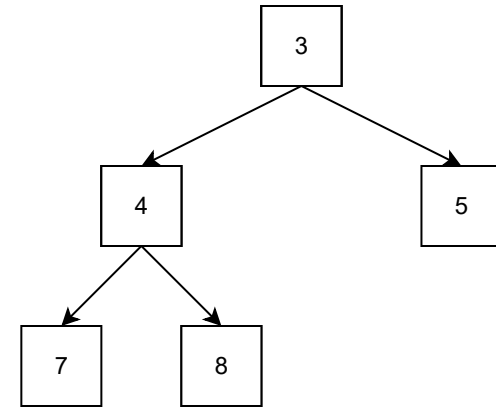
上の段から順番に、左から順番に埋まる。

親の値は子の値以下

ヒープの高さはおよそ $\log(n)$ となるので、

push / pop が $O(\log(n))$ で実装できる

データを全部ヒープに入れて、
小さい方から全部取り出すと、
 $O(n\log(n))$ でソートができる。



1を pop。

上から順に詰まるルールを満たしたい。

一番右下の8を削除し、一番上に。

8は3,5より大きい。

8と小さい3を交換。

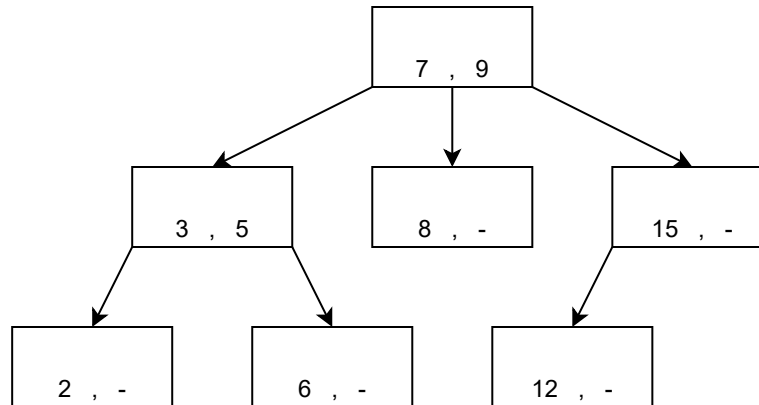
8は4,7より小さい。

8と小さい4を交換。

COLLECTION - 大小で整理

探索木 (SEARCH TREE)

追加_(add)・削除_(delete)に加え、探索_(find)を効率的にしたい。



N-分木 (子供が最大N個)で、ノードは n-1個の数字を持ち、全てのノードが以下を満たす。

0番目子供以下の全部の数字 < 0番目の子供の数字

< 1番目子供以下の全部の数字

< 1番目の子供の数字

...

< N番目子供以下の全部の数字

特に二分木_(binary tree)のものを二分探索木_(Binary Search Tree)と呼ぶ。

平衡二分木

左右の高さのバランスが、
良いと探索に $O(\log(n))$

悪いと探索に $O(n)$

追加/削除時に効率良くバランス調整を
したい。

AVL木

データ追加・削除時に、
左右の高さが2以上ずれたら木を修正。

追加・削除・探索共に $O(\log(n))$

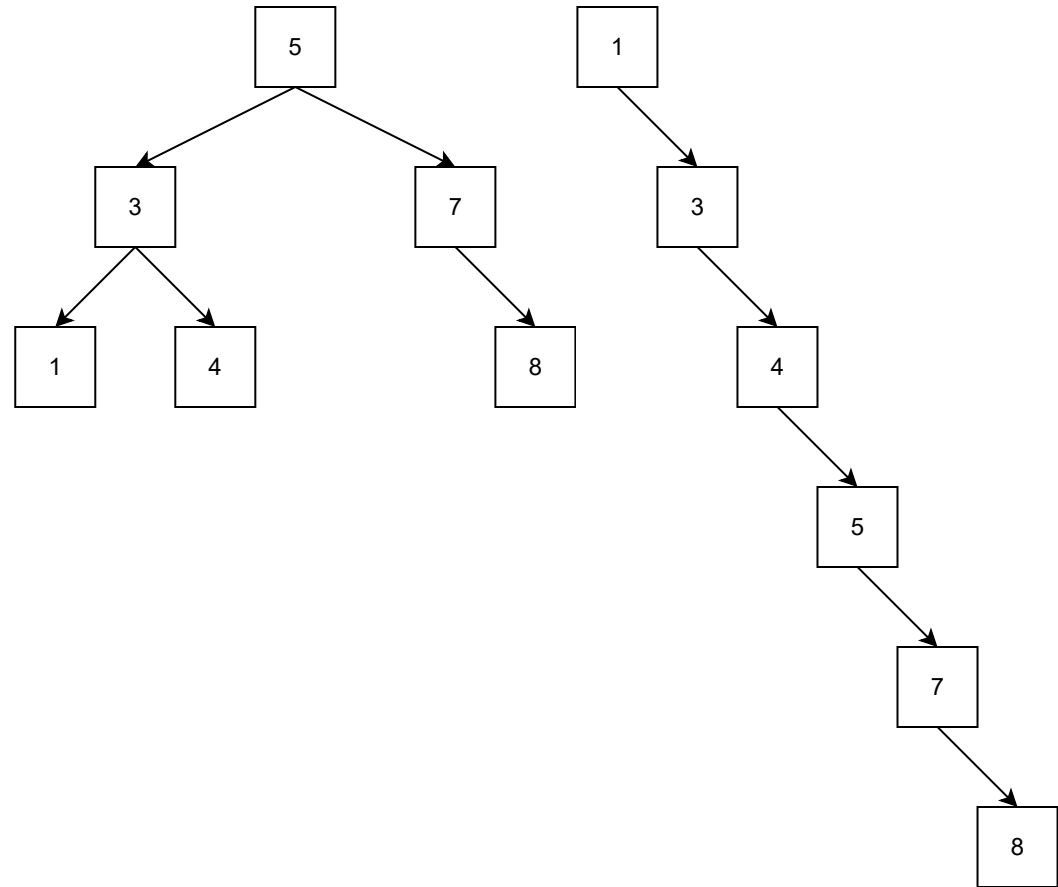
赤黒木

データ追加・削除時に、
左右の高さが2倍はずれないよう木を修
正。

追加・削除・探索共に $O(\log(n))$

(追加・削除時のリバランスは $O(1)$)

AVL 木より効率的



COLLECTION - 同じかどうかで整理

ハッシュテーブル

ハッシュ関数

(hash: 切り刻ざんで混ぜる。Hashed Potato, etc..)

$Hash(A) = B$ としたとき、
Bの値の出現確率が一樣になるもの。

暗号学的ハッシュ関数

- Hash(A)からAを推測できない
- 同じHash(A)となる
二つのAを見つけるのが難しい
- Aを少し返ると全く違うBになる

sha-256("あ") =
ae0e541ce66494516f5c9cfd9cd33084
94ddc5a4d3d82f7e15792959dc661538

ハッシュテーブル (HASH TABLE)

0	1	2	3	4	5	6	7
	17						

Hash(17) = 17 % 8 = 1

動的配列を使う。

A を、Hash(A) % Capacity の位置に入れる。
既に他のモノが入っていたら、何とかする
(ハッシュの衝突・後述)

ハッシュの衝突

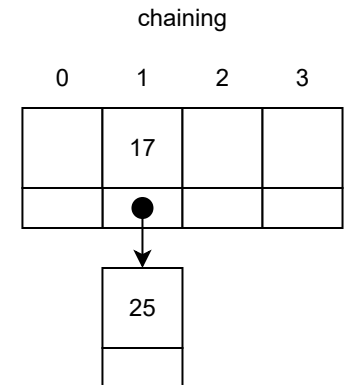
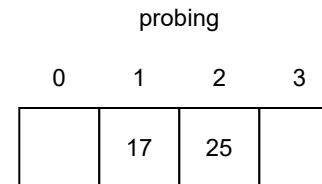
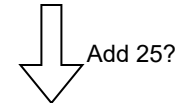
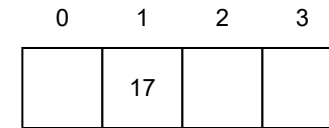
衝突対策

プロービング法_(probing)

とりあえず次の空きマスに入れる。
削除時には墓石を建て、次回探索時に備える。

チェイン法_(chaining)

衝突したものをリンクリストで保持



計算効率

Capacity: C, Size: S

使用率 $p = C / S$

一回の操作で $1/p$ 個アクセスが必要

空きが多い($p \approx 0$)と、ほぼ 1

空きが少ない($p \approx 1$)と、ほぼ N

80%程度埋まったらCapacityを増やす_(rehash)

追加・削除が償却 $O(1)$ ・ 探索が $O(1)$

マップ/セット

マップ (MAP / 写像)

キー (key) と値 (value) のペアを保持。

キーの重複は不可。 (許すバージョンは MultiMap と呼ぶ)

キーに順序を付けたい場合は探索木

キーの順序が不要な場合はハッシュテーブル
で実装。

(それぞれ、Ordered Map, Unordered Map と呼ぶ)

セット (SET / 集合)

マップのキーだけバージョンと等価

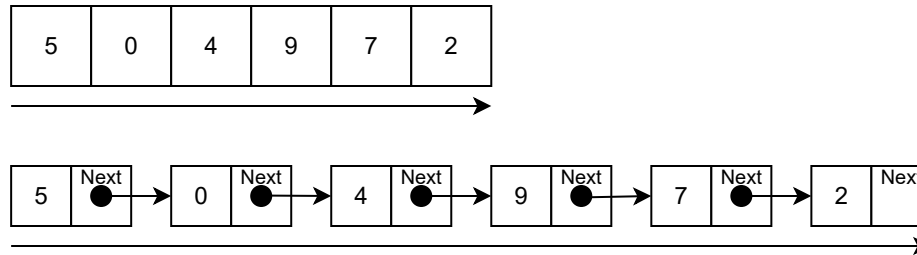
(同様に、MutiSet, Ordered Set, Unordered Setがある)

```
map[Key1] = Value1;
map[Key2] = Value2;
map[Key1] = Value3;
print map[Key2]
// Value2
print map[Key1]
// Value3

set.insert(4);
set.insert(9);
print set.find(4);
// true
print set.find(7);
// false
```


探索アルゴリズム - SEARCH ALGORITHM

線形探索



(別名: 総当たり、馬鹿探索 (brute force))

前から順番に見ていく。いつでも使える。計算時間: $O(n)$

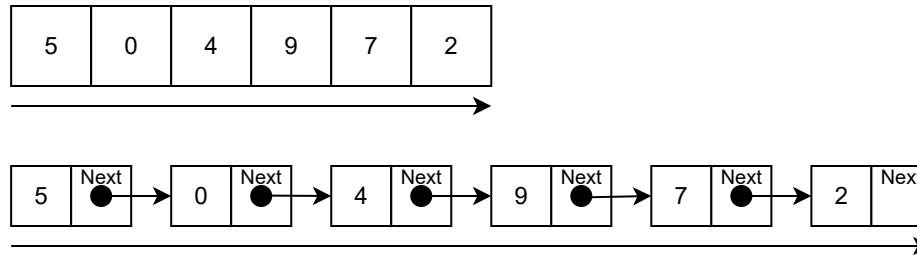
二分探索 (BINARY SEARCH)

Find: 7

0	1	2	3	4	5	6	7	8
0	1	3	5	6	7	9	10	15

半分ずつに絞り込む。ソート済みじゃないと使えない。計算時間: $O(\log(n))$

線形探索



(別名: 総当たり、馬鹿探索 (brute force))

前から順番に見ていく。いつでも使える。計算時間: $O(n)$

二分探索 (BINARY SEARCH)

Find: 7

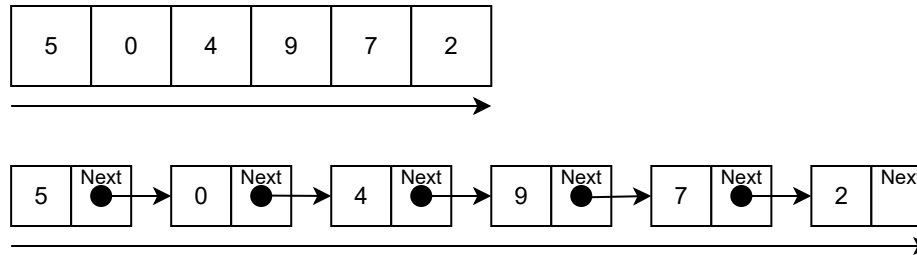
0	1	2	3	4	5	6	7	8
0	1	3	5	6	7	9	10	15

7を探そう。可能性があるのは、index: 0以上9未満。

L:0, R:9

半分ずつに絞り込む。ソート済みじゃないと使えない。計算時間: $O(\log(n))$

線形探索



(別名: 総当たり、馬鹿探索 (brute force))

前から順番に見ていく。いつでも使える。計算時間: $O(n)$

二分探索 (BINARY SEARCH)

Find: 7

0	1	2	3	4	5	6	7	8
0	1	3	5	6	7	9	10	15

7を探そう。可能性があるのは、index: 0以上9未満。

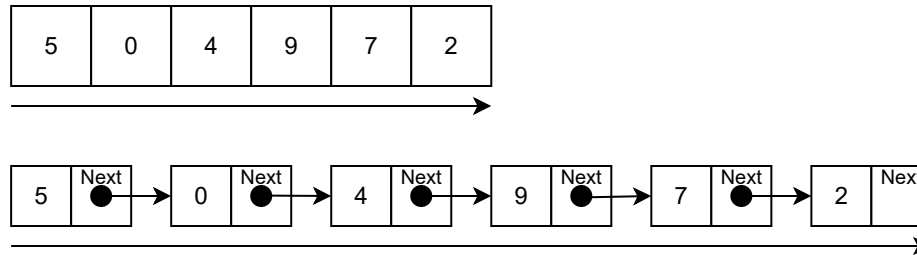
真ん中のindex:4は6で7より小さい。可能性は index: 5以上9未満。

L:0, R:9

L:5, R:9

半分ずつに絞り込む。ソート済みじゃないと使えない。計算時間: $O(\log(n))$

線形探索

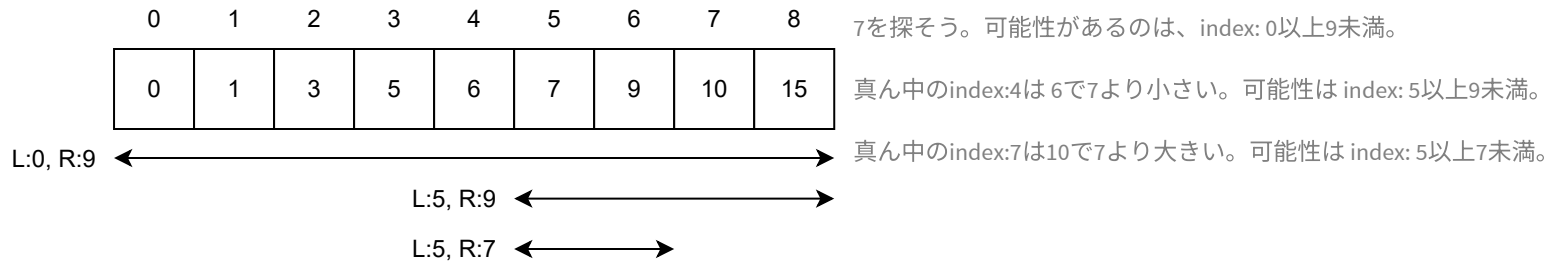


(別名: 総当たり、馬鹿探索 (brute force))

前から順番に見ていく。いつでも使える。計算時間: $O(n)$

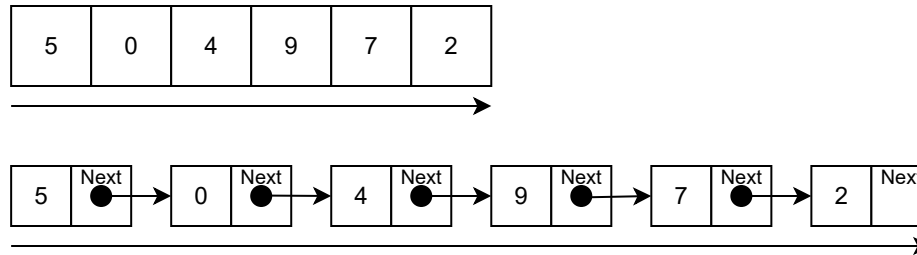
二分探索 (BINARY SEARCH)

Find: 7



半分ずつに絞り込む。ソート済みじゃないと使えない。計算時間: $O(\log(n))$

線形探索

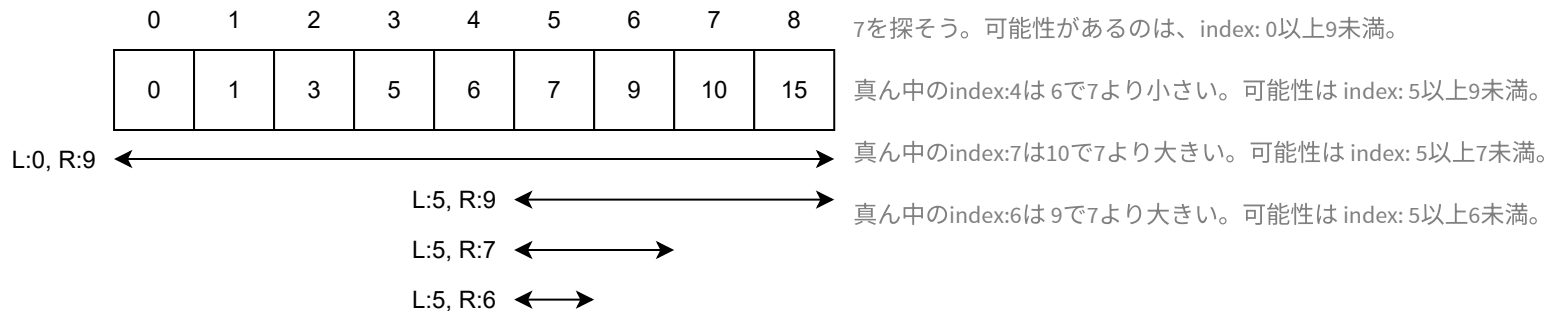


(別名: 総当たり、馬鹿探索 (brute force))

前から順番に見ていく。いつでも使える。計算時間: $O(n)$

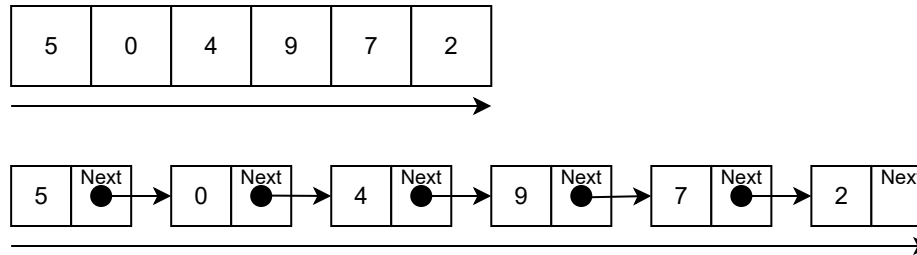
二分探索 (BINARY SEARCH)

Find: 7



半分ずつに絞り込む。ソート済みじゃないと使えない。計算時間: $O(\log(n))$

線形探索

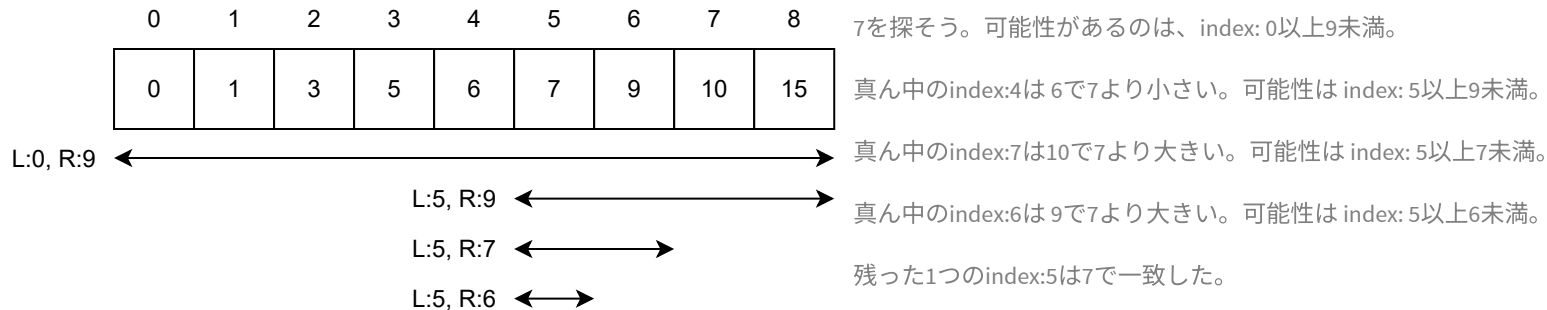


(別名: 総当たり、馬鹿探索 (brute force))

前から順番に見ていく。いつでも使える。計算時間: $O(n)$

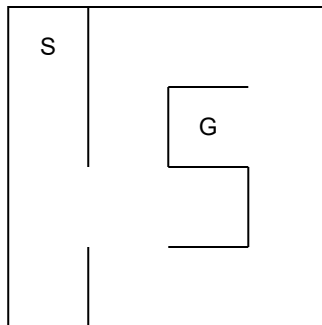
二分探索 (BINARY SEARCH)

Find: 7

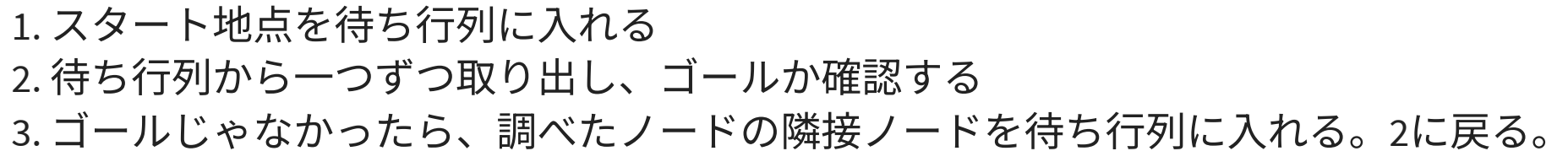


半分ずつに絞り込む。ソート済みじゃないと使えない。計算時間: $O(\log(n))$

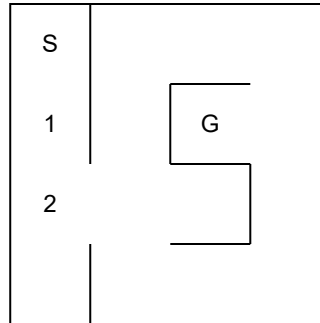
幅優先探索(BFS)



1. スタート地点を待ち行列に入れる
2. 待ち行列から一つずつ取り出し、ゴールか確認する
3. ゴールじゃなかったら、調べたノードの隣接ノードを待ち行列に入れる。2に戻る。

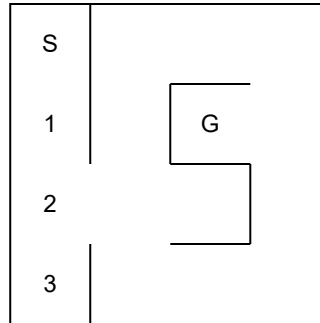


幅優先探索(BFS)



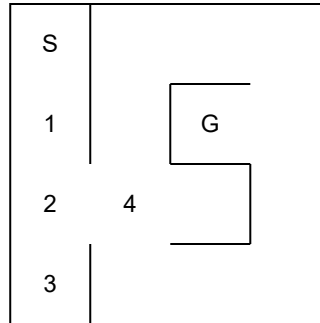
1. スタート地点を待ち行列に入れる
2. 待ち行列から一つずつ取り出し、ゴールか確認する
3. ゴールじゃなかったら、調べたノードの隣接ノードを待ち行列に入れる。2に戻る。

幅優先探索(BFS)



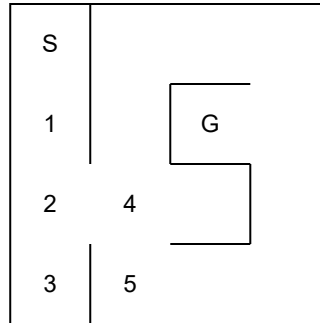
1. スタート地点を待ち行列に入れる
2. 待ち行列から一つずつ取り出し、ゴールか確認する
3. ゴールじゃなかったら、調べたノードの隣接ノードを待ち行列に入れる。2に戻る。

幅優先探索(BFS)



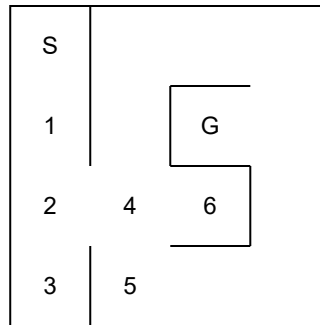
1. スタート地点を待ち行列に入れる
2. 待ち行列から一つずつ取り出し、ゴールか確認する
3. ゴールじゃなかったら、調べたノードの隣接ノードを待ち行列に入れる。2に戻る。

幅優先探索(BFS)



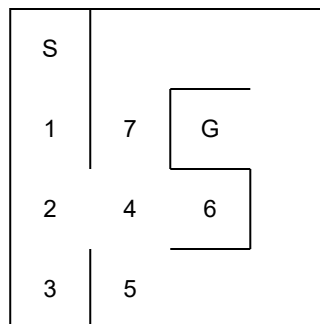
1. スタート地点を待ち行列に入れる
2. 待ち行列から一つずつ取り出し、ゴールか確認する
3. ゴールじゃなかったら、調べたノードの隣接ノードを待ち行列に入れる。2に戻る。

幅優先探索(BFS)



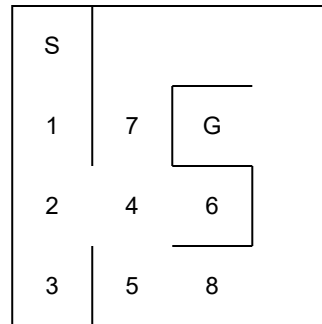
1. スタート地点を待ち行列に入れる
2. 待ち行列から一つずつ取り出し、ゴールか確認する
3. ゴールじゃなかったら、調べたノードの隣接ノードを待ち行列に入れる。2に戻る。

幅優先探索(BFS)



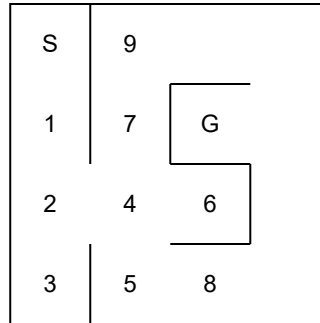
1. スタート地点を待ち行列に入れる
2. 待ち行列から一つずつ取り出し、ゴールか確認する
3. ゴールじゃなかったら、調べたノードの隣接ノードを待ち行列に入れる。2に戻る。

幅優先探索(BFS)



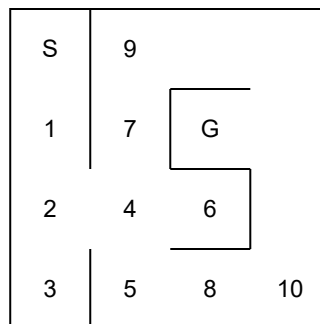
1. スタート地点を待ち行列に入れる
2. 待ち行列から一つずつ取り出し、ゴールか確認する
3. ゴールじゃなかったら、調べたノードの隣接ノードを待ち行列に入れる。2に戻る。

幅優先探索(BFS)



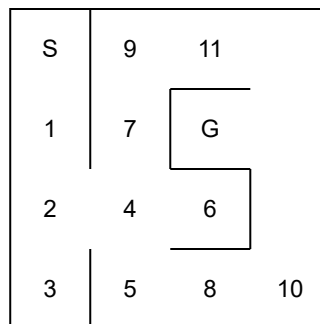
1. スタート地点を待ち行列に入れる
2. 待ち行列から一つずつ取り出し、ゴールか確認する
3. ゴールじゃなかったら、調べたノードの隣接ノードを待ち行列に入れる。2に戻る。

幅優先探索(BFS)



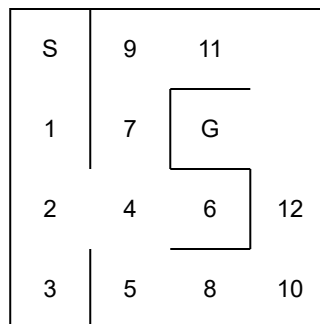
1. スタート地点を待ち行列に入れる
2. 待ち行列から一つずつ取り出し、ゴールか確認する
3. ゴールじゃなかったら、調べたノードの隣接ノードを待ち行列に入れる。2に戻る。

幅優先探索(BFS)



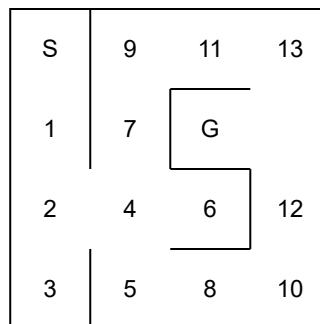
1. スタート地点を待ち行列に入れる
2. 待ち行列から一つずつ取り出し、ゴールか確認する
3. ゴールじゃなかったら、調べたノードの隣接ノードを待ち行列に入れる。2に戻る。

幅優先探索(BFS)



1. スタート地点を待ち行列に入れる
2. 待ち行列から一つずつ取り出し、ゴールか確認する
3. ゴールじゃなかったら、調べたノードの隣接ノードを待ち行列に入れる。2に戻る。

幅優先探索(BFS)



1. スタート地点を待ち行列に入れる
2. 待ち行列から一つずつ取り出し、ゴールか確認する
3. ゴールじゃなかったら、調べたノードの隣接ノードを待ち行列に入れる。2に戻る。

幅優先探索(BFS)

S	9	11	13
1	7	G	14
2	4	6	12
3	5	8	10

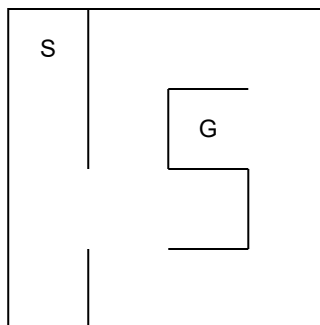
1. スタート地点を待ち行列に入れる
2. 待ち行列から一つずつ取り出し、ゴールか確認する
3. ゴールじゃなかったら、調べたノードの隣接ノードを待ち行列に入れる。2に戻る。

幅優先探索(BFS)

S	9	11	13
1	7	15 G	14
2	4	6	12
3	5	8	10

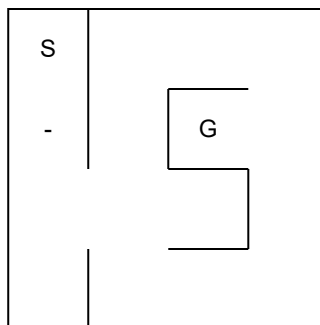
1. スタート地点を待ち行列に入れる
2. 待ち行列から一つずつ取り出し、ゴールか確認する
3. ゴールじゃなかったら、調べたノードの隣接ノードを待ち行列に入れる。2に戻る。

深さ優先探索(DFS)

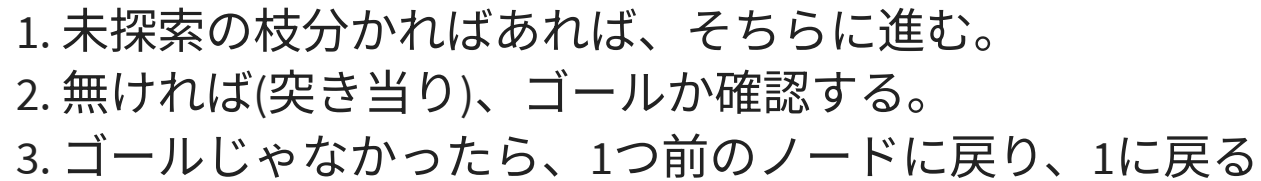


1. 未探索の枝分かればあれば、そちらに進む。
2. 無ければ(突き当り)、ゴールか確認する。
3. ゴールじゃなかったら、1つ前のノードに戻り、1に戻る

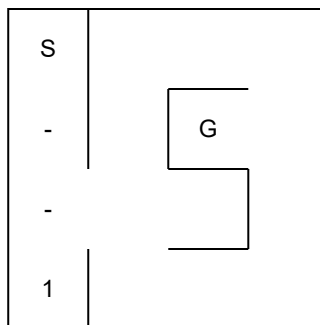
深さ優先探索(DFS)



1. 未探索の枝分かればあれば、そちらに進む。
2. 無ければ(突き当り)、ゴールか確認する。
3. ゴールじゃなかったら、1つ前のノードに戻り、1に戻る

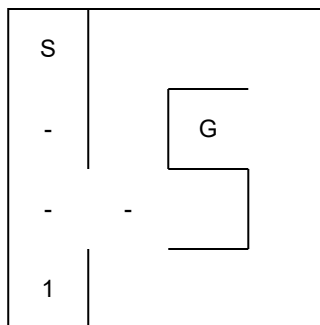


深さ優先探索(DFS)



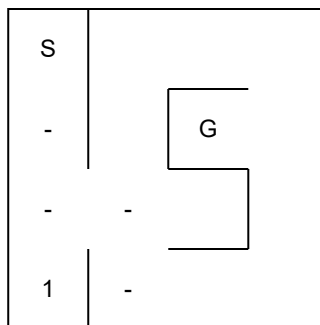
1. 未探索の枝分かればあれば、そちらに進む。
2. 無ければ(突き当り)、ゴールか確認する。
3. ゴールじゃなかったら、1つ前のノードに戻り、1に戻る

深さ優先探索(DFS)



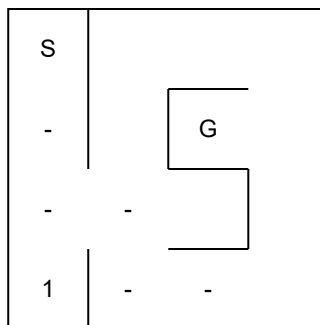
1. 未探索の枝分かればあれば、そちらに進む。
2. 無ければ(突き当り)、ゴールか確認する。
3. ゴールじゃなかったら、1つ前のノードに戻り、1に戻る

深さ優先探索(DFS)



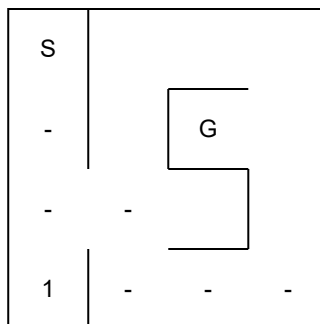
1. 未探索の枝分かればあれば、そちらに進む。
2. 無ければ(突き当り)、ゴールか確認する。
3. ゴールじゃなかったら、1つ前のノードに戻り、1に戻る

深さ優先探索(DFS)



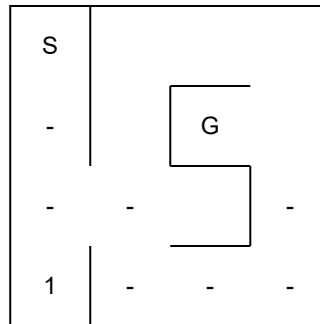
1. 未探索の枝分かればあれば、そちらに進む。
2. 無ければ(突き当り)、ゴールか確認する。
3. ゴールじゃなかったら、1つ前のノードに戻り、1に戻る

深さ優先探索(DFS)



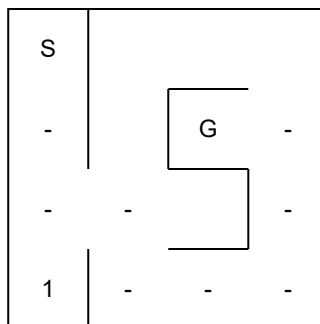
1. 未探索の枝分かれがあれば、そちらに進む。
2. 無ければ(突き当り)、ゴールか確認する。
3. ゴールじゃなかったら、1つ前のノードに戻り、1に戻る

深さ優先探索(DFS)



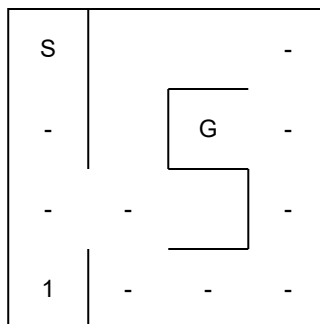
1. 未探索の枝分かればあれば、そちらに進む。
2. 無ければ(突き当り)、ゴールか確認する。
3. ゴールじゃなかったら、1つ前のノードに戻り、1に戻る

深さ優先探索(DFS)



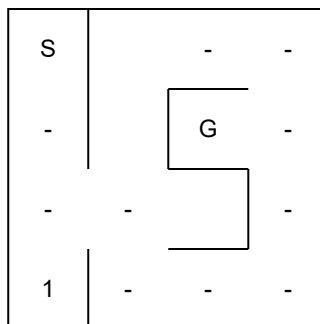
1. 未探索の枝分かればあれば、そちらに進む。
2. 無ければ(突き当り)、ゴールか確認する。
3. ゴールじゃなかったら、1つ前のノードに戻り、1に戻る

深さ優先探索(DFS)



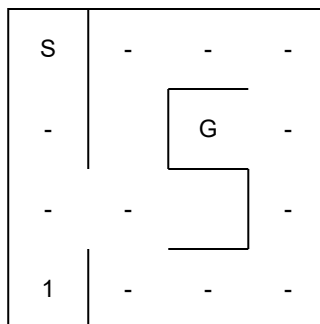
1. 未探索の枝分かればあれば、そちらに進む。
2. 無ければ(突き当り)、ゴールか確認する。
3. ゴールじゃなかったら、1つ前のノードに戻り、1に戻る

深さ優先探索(DFS)



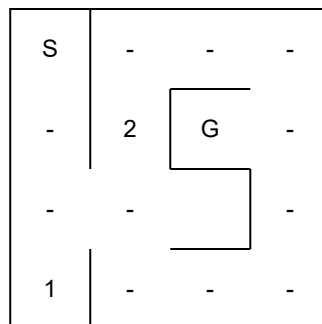
1. 未探索の枝分かればあれば、そちらに進む。
2. 無ければ(突き当り)、ゴールか確認する。
3. ゴールじゃなかったら、1つ前のノードに戻り、1に戻る

深さ優先探索(DFS)

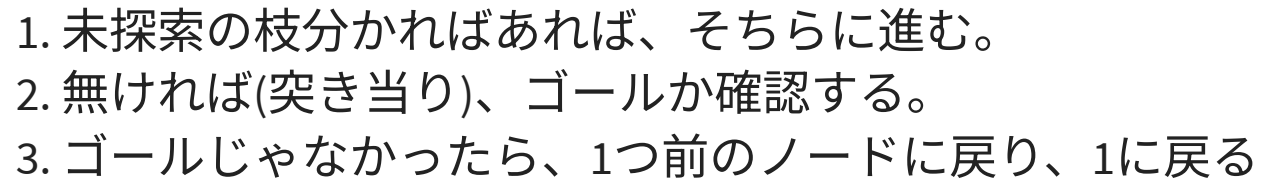


1. 未探索の枝分かればあれば、そちらに進む。
2. 無ければ(突き当り)、ゴールか確認する。
3. ゴールじゃなかったら、1つ前のノードに戻り、1に戻る

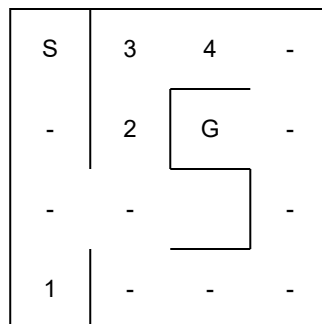
深さ優先探索(DFS)



1. 未探索の枝分かればあれば、そちらに進む。
2. 無ければ(突き当り)、ゴールか確認する。
3. ゴールじゃなかったら、1つ前のノードに戻り、1に戻る

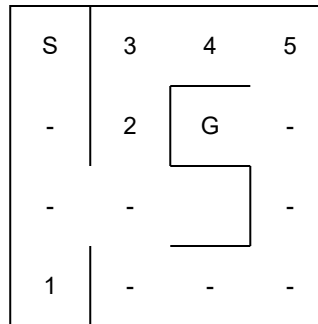


深さ優先探索(DFS)



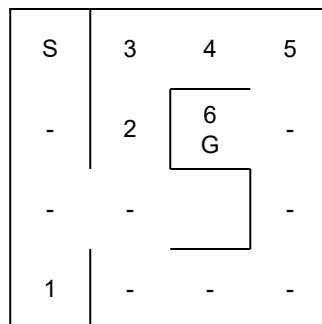
1. 未探索の枝分かればあれば、そちらに進む。
2. 無ければ(突き当り)、ゴールか確認する。
3. ゴールじゃなかったら、1つ前のノードに戻り、1に戻る

深さ優先探索(DFS)



1. 未探索の枝分かればあれば、そちらに進む。
2. 無ければ(突き当り)、ゴールか確認する。
3. ゴールじゃなかったら、1つ前のノードに戻り、1に戻る

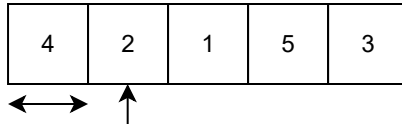
深さ優先探索(DFS)



1. 未探索の枝分かればあれば、そちらに進む。
2. 無ければ(突き当り)、ゴールか確認する。
3. ゴールじゃなかったら、1つ前のノードに戻り、1に戻る

整列アルゴリズム - SORT ALGORITHM

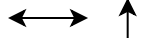
挿入ソート (INSERTION SORT)



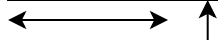
1. index:0から0は整列済み、index:1の2の加えたい。
入れる場所を順に探し、後ろを1つずつにずらす。

挿入ソート (INSERTION SORT)

4	2	1	5	3
---	---	---	---	---



2	4	1	5	3
---	---	---	---	---



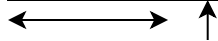
1. index:0から0は整列済み、index:1の2の加えたい。
入れる場所を順に探し、後ろを1つずつにずらす。
2. index:0から1は整列済み、index:2の1の加えたい。
入れる場所を順に探し、後ろを1つずつにずらす。

挿入ソート (INSERTION SORT)

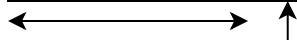
4	2	1	5	3
---	---	---	---	---



2	4	1	5	3
---	---	---	---	---



1	2	4	5	3
---	---	---	---	---



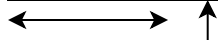
1. index:0から0は整列済み、index:1の2の加えたい。
入れる場所を順に探し、後ろを1つずつにずらす。
2. index:0から1は整列済み、index:2の1の加えたい。
入れる場所を順に探し、後ろを1つずつにずらす。
3. index:0から2は整列済み、index:3の4の加えたい。
入れる場所を順に探し、後ろを1つずつにずらす。

挿入ソート (INSERTION SORT)

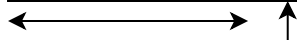
4	2	1	5	3
---	---	---	---	---



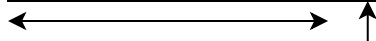
2	4	1	5	3
---	---	---	---	---



1	2	4	5	3
---	---	---	---	---



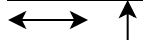
1	2	4	5	3
---	---	---	---	---



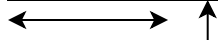
1. index:0から0は整列済み、index:1の2の加えたい。
入れる場所を順に探し、後ろを1つずつにずらす。
2. index:0から1は整列済み、index:2の1の加えたい。
入れる場所を順に探し、後ろを1つずつにずらす。
3. index:0から2は整列済み、index:3の4の加えたい。
入れる場所を順に探し、後ろを1つずつにずらす。
4. index:0から3は整列済み、index:4の3の加えたい。
入れる場所を順に探し、後ろを1つずつにずらす。

挿入ソート (INSERTION SORT)

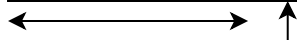
4	2	1	5	3
---	---	---	---	---



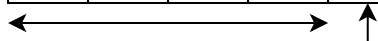
2	4	1	5	3
---	---	---	---	---



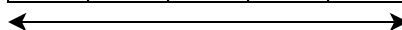
1	2	4	5	3
---	---	---	---	---



1	2	4	5	3
---	---	---	---	---



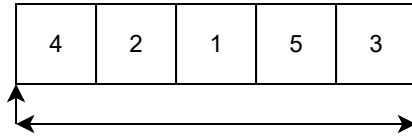
1	2	3	4	5
---	---	---	---	---



1. index:0から0は整列済み、index:1の2の加えたい。
入れる場所を順に探し、後ろを1つずつにずらす。
2. index:0から1は整列済み、index:2の1の加えたい。
入れる場所を順に探し、後ろを1つずつにずらす。
3. index:0から2は整列済み、index:3の4の加えたい。
入れる場所を順に探し、後ろを1つずつにずらす。
4. index:0から3は整列済み、index:4の3の加えたい。
入れる場所を順に探し、後ろを1つずつにずらす。

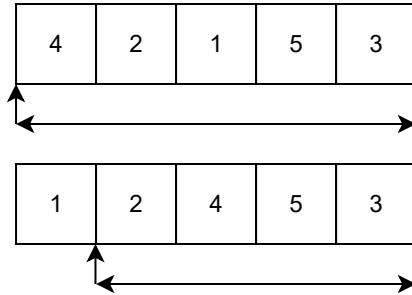
計算時間は $O(n^2)$

選択ソート (SELECTION SORT)



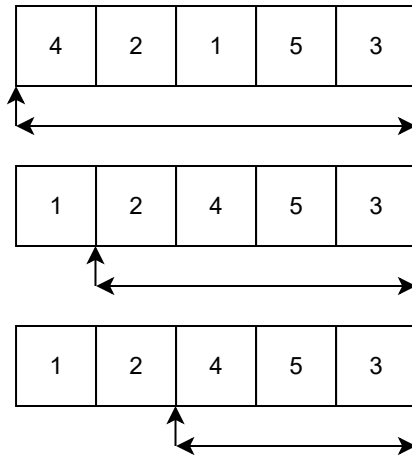
1. index:0に入る、一番小さな数は?
index:0-4を探して、見つけたらindex:0と交換。

選択ソート (SELECTION SORT)



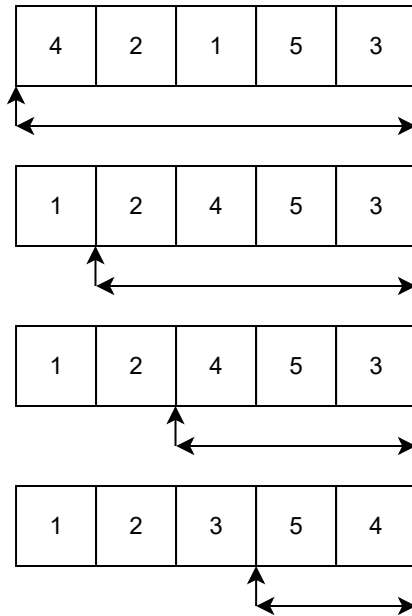
1. index:0に入る、一番小さな数は?
index:0-4を探して、見つけたらindex:0と交換。
2. index:1に入る、次に小さな数は?
index:1-4を探して、見つけたらindex:1と交換。

選択ソート (SELECTION SORT)



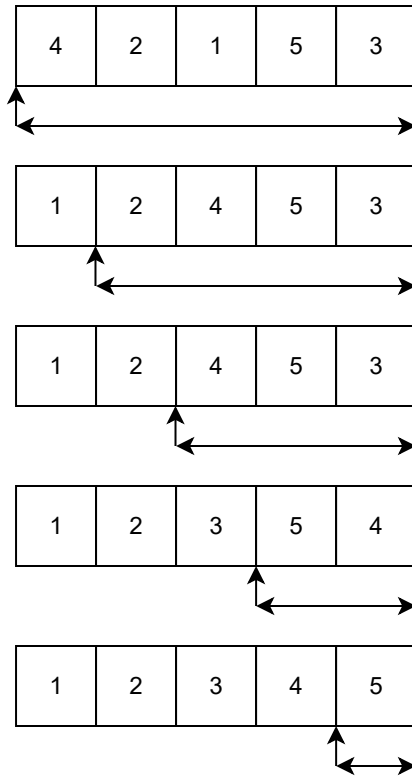
1. index:0に入る、一番小さな数は?
index:0-4を探して、見つけたらindex:0と交換。
2. index:1に入る、次に小さな数は?
index:1-4を探して、見つけたらindex:1と交換。
3. index:2に入る、次に小さな数は?
index:2-4を探して、見つけたらindex:2と交換。

選択ソート (SELECTION SORT)



1. index:0に入る、一番小さな数は?
index:0-4を探して、見つけたらindex:0と交換。
2. index:1に入る、次に小さな数は?
index:1-4を探して、見つけたらindex:1と交換。
3. index:2に入る、次に小さな数は?
index:2-4を探して、見つけたらindex:2と交換。
4. index:3に入る、次に小さな数は?
index:3-4を探して、見つけたらindex:3と交換。

選択ソート (SELECTION SORT)

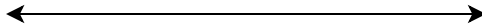


1. index:0に入る、一番小さな数は?
index:0-4を探して、見つけたらindex:0と交換。
2. index:1に入る、次に小さな数は?
index:1-4を探して、見つけたらindex:1と交換。
3. index:2に入る、次に小さな数は?
index:2-4を探して、見つけたらindex:2と交換。
4. index:3に入る、次に小さな数は?
index:3-4を探して、見つけたらindex:3と交換。

計算時間は $O(n^2)$

クイックソート (QUICK SORT)

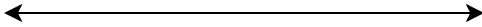
4	2	1	6	3	5
---	---	---	---	---	---



1. index:0-4が未整列。
基準値(pivot)を選ぶ(今回は4)
それより小さいのを前へ寄せ、
それより大きいのを後ろへ寄せる。

クイックソート (QUICK SORT)

4	2	1	6	3	5
---	---	---	---	---	---



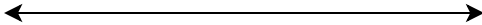
2	1	3	4	6	5
---	---	---	---	---	---



1. index:0-4が未整列。
基準値(pivot)を選ぶ(今回は4)
それより小さいのを前へ寄せ、
それより大きいのを後ろへ寄せる。
2. index:0-2の小さい組、
index:4-5の大きい組は未整列。
小さい組をクイックソートする。

クイックソート (QUICK SORT)

4	2	1	6	3	5
---	---	---	---	---	---



2	1	3	4	6	5
---	---	---	---	---	---

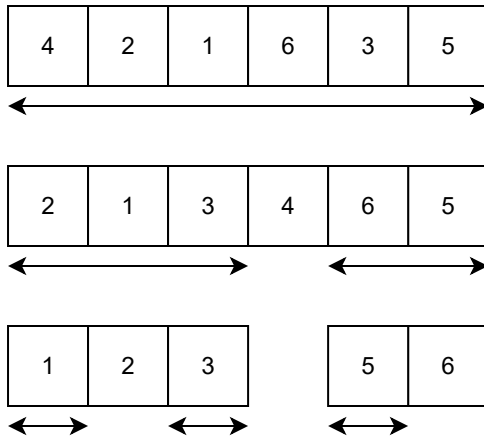


1	2	3
---	---	---



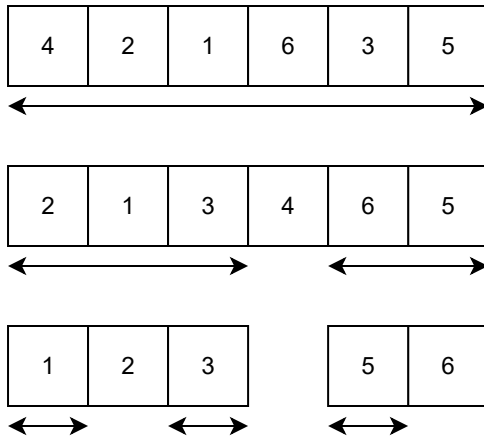
1. index:0-4が未整列。
基準値(pivot)を選ぶ(今回は4)
それより小さいのを前へ寄せ、
それより大きいのを後ろへ寄せる。
2. index:0-2の小さい組、
index:4-5の大きい組は未整列。
小さい組をクイックソートする。
3. index:0-2を基準値2で整列。
小さい組は残り1つなので整列完了。
大きい組も残り1つなので整列完了。
index:0-2は終わったので、次はindex:4-5

クイックソート (QUICK SORT)



1. index:0-4が未整列。
基準値(pivot)を選ぶ(今回は4)
それより小さいのを前へ寄せ、
それより大きいのを後ろへ寄せる。
2. index:0-2の小さい組、
index:4-5の大きい組は未整列。
小さい組をクイックソートする。
3. index:0-2を基準値2で整列。
小さい組は残り1つなので整列完了。
大きい組も残り1つなので整列完了。
index:0-2は終わったので、次はindex:4-5
4. index:4-5を基準値6で整列。
小さい組は残り1つなので整列完了。
大きい組は無かったので整列完了。

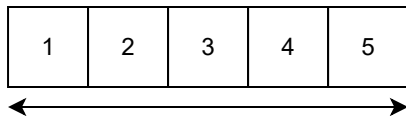
クイックソート (QUICK SORT)



1. index:0-4が未整列。
基準値(pivot)を選ぶ(今回は4)
それより小さいのを前へ寄せ、
それより大きいのを後ろへ寄せる。
2. index:0-2の小さい組、
index:4-5の大きい組は未整列。
小さい組をクイックソートする。
3. index:0-2を基準値2で整列。
小さい組は残り1つなので整列完了。
大きい組も残り1つなので整列完了。
index:0-2は終わったので、次はindex:4-5
4. index:4-5を基準値6で整列。
小さい組は残り1つなので整列完了。
大きい組は無かったので整列完了。

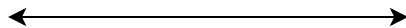
平均の深さが $\log(n)$ となるので、
平均計算時間は、 $O(n\log(n))$

クイックソート (QUICK SORT)

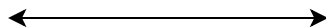


クイックソート (QUICK SORT)

1	2	3	4	5
---	---	---	---	---

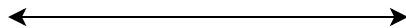


1	2	3	4	5
---	---	---	---	---

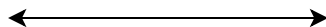


クイックソート (QUICK SORT)

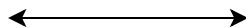
1	2	3	4	5
---	---	---	---	---



1	2	3	4	5
---	---	---	---	---

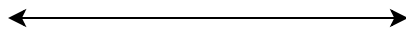


2	3	4	5
---	---	---	---

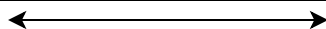


クイックソート (QUICK SORT)

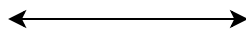
1	2	3	4	5
---	---	---	---	---



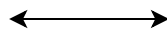
1	2	3	4	5
---	---	---	---	---



2	3	4	5
---	---	---	---

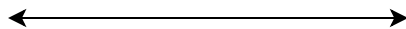


3	4	5
---	---	---

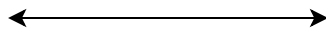


クイックソート (QUICK SORT)

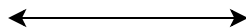
1	2	3	4	5
---	---	---	---	---



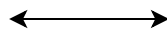
1	2	3	4	5
---	---	---	---	---



2	3	4	5
---	---	---	---



3	4	5
---	---	---

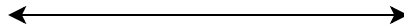


4	5
---	---

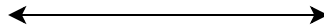


クイックソート (QUICK SORT)

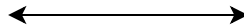
1	2	3	4	5
---	---	---	---	---



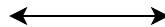
1	2	3	4	5
---	---	---	---	---



2	3	4	5
---	---	---	---



3	4	5
---	---	---



4	5
---	---



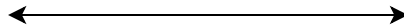
pivot の選び方が悪いと、分割が上手くいかず、
最悪計算時間は、 $O(n^2)$

できるだけ中央値に近い値を選ぶのが良いが、
中央値の計算に時間がかかると本末転倒。
3ヶ所の中央値を使うなどの工夫をする。

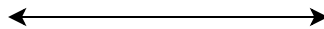
他にも、stack overflow を避けるために、短い方を再帰呼び出しし、長い方をループで処理する等の工夫がされる。

クイックソート (QUICK SORT)

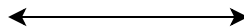
1	2	3	4	5
---	---	---	---	---



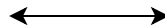
1	2	3	4	5
---	---	---	---	---



2	3	4	5
---	---	---	---



3	4	5
---	---	---



4	5
---	---

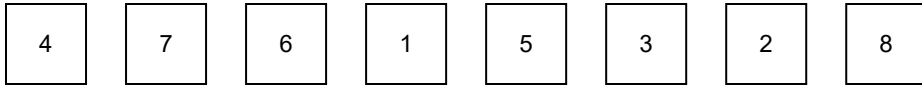


pivot の選び方が悪いと、分割が上手くいかず、
最悪計算時間は、 $O(n^2)$

できるだけ中央値に近い値を選ぶのが良いが、
中央値の計算に時間がかかると本末転倒。
3ヶ所の中央値を使うなどの工夫をする。

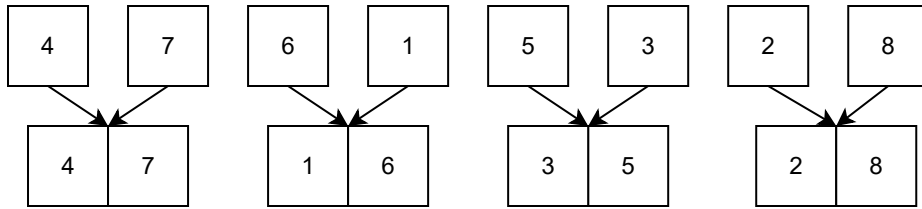
他にも、stack overflow を避けるために、短い方を再帰呼び出しし、長い方をループで処理する等の工夫がされる。

マージソート (MERGE SORT)



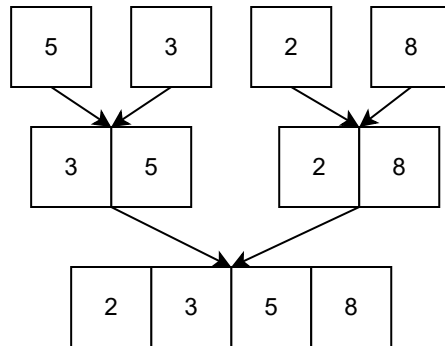
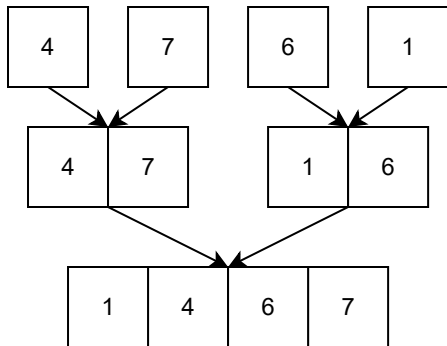
1. 長さ8の未整列のデータがある。
もとい、長さ1の整列データが8個ある。

マージソート (MERGE SORT)



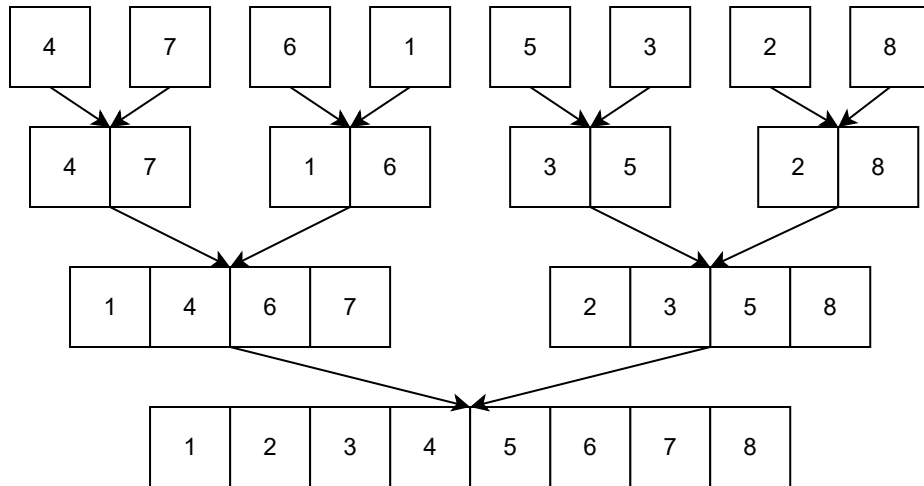
1. 長さ8の未整列のデータがある。
もとい、長さ1の整列データが8個ある。
2. 長さ1の整列データを2つずつマージ
長さ2の整列データが4つある。

マージソート (MERGE SORT)



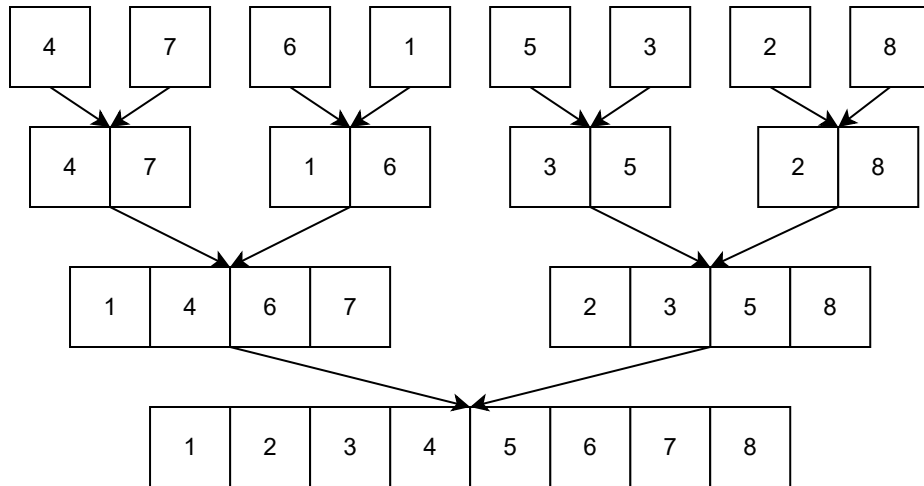
1. 長さ8の未整列のデータがある。
もとい、長さ1の整列データが8個ある。
2. 長さ1の整列データを2つずつマージ
長さ2の整列データが4つある。
3. 長さ2の整列データを2つずつマージ
整列データのマージは、
2つの列の先頭から小さい方を取り出すのを
繰り返す。長さ4の整列データが2つある。

マージソート (MERGE SORT)



1. 長さ8の未整列のデータがある。
もとい、長さ1の整列データが8個ある。
2. 長さ1の整列データを2つずつマージ
長さ2の整列データが4つある。
3. 長さ2の整列データを2つずつマージ
整列データのマージは、
2つの列の先頭から小さい方を取り出すのを
繰り返す。長さ4の整列データが2つある。
4. 長さ4の整列データを2つマージ
長さ8の整列データができた。

マージソート (MERGE SORT)

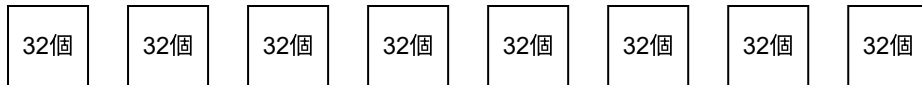


1. 長さ8の未整列のデータがある。
もとい、長さ1の整列データが8個ある。
2. 長さ1の整列データを2つずつマージ
長さ2の整列データが4つある。
3. 長さ2の整列データを2つずつマージ
整列データのマージは、
2つの列の先頭から小さい方を取り出すのを
繰り返す。長さ4の整列データが2つある。
4. 長さ4の整列データを2つマージ
長さ8の整列データができた。

平均計算時間、最悪計算時間共に
 $O(n \log(n))$

(うちの2歳も大好き)

ティムソート (TIM SORT)

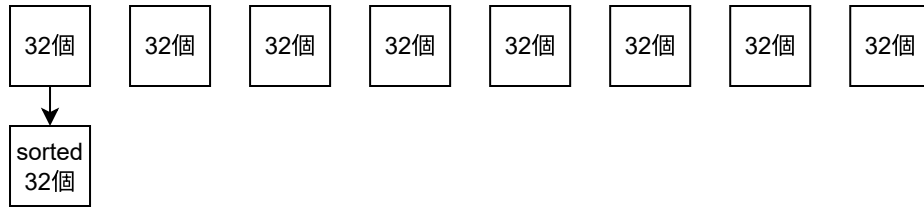


データを32個ずつに分割し、
32個のデータは挿入ソートで整列、
それらをマージしていく。

32個までは merge sort より insertion sort の方が速い

CPU キャッシュに当たりやすい順番で行う。

ティムソート (TIM SORT)

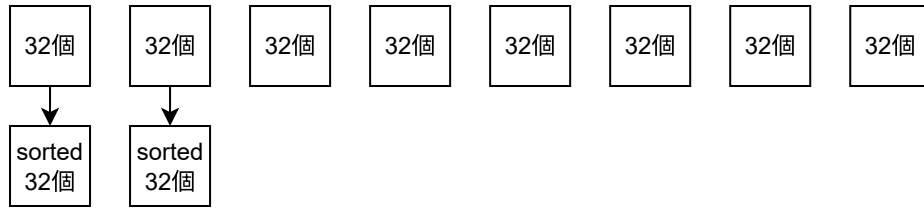


データを32個ずつに分割し、
32個のデータは挿入ソートで整列、
それらをマージしていく。

32個までは merge sort より insertion sort の方が速い

CPU キャッシュに当たりやすい順番で行う。

ティムソート (TIM SORT)

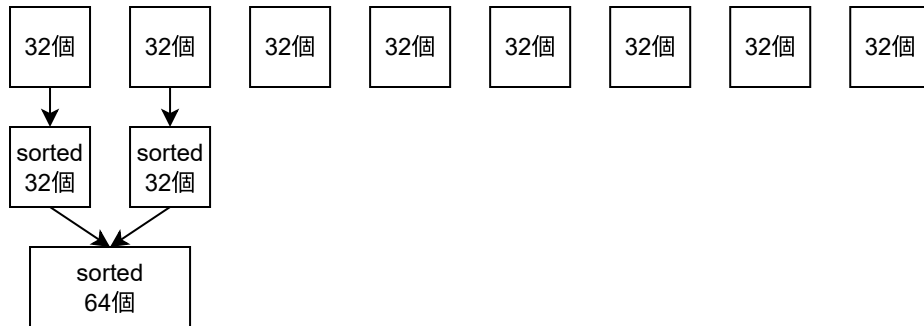


データを32個ずつに分割し、
32個のデータは挿入ソートで整列、
それらをマージしていく。

32個までは merge sort より insertion sort の方が速い

CPU キャッシュに当たりやすい順番で行う。

ティムソート (TIM SORT)

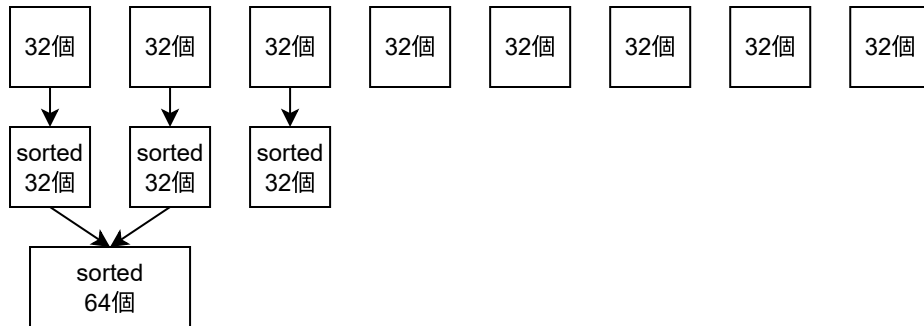


データを32個ずつに分割し、
32個のデータは挿入ソートで整列、
それらをマージしていく。

32個までは merge sort より insertion sort の方が速い

CPU キャッシュに当たりやすい順番で行う。

ティムソート (TIM SORT)

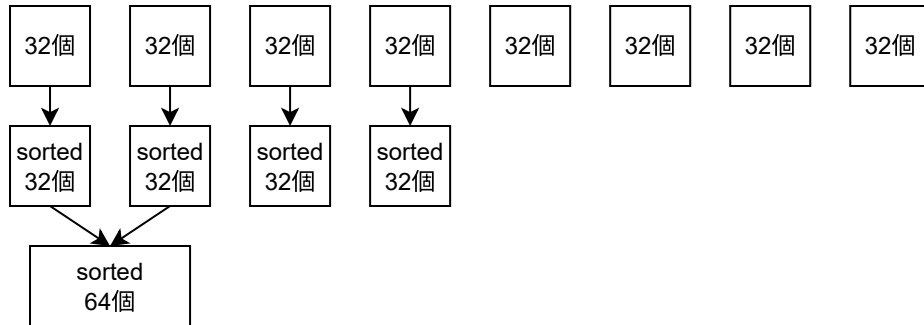


データを32個ずつに分割し、
32個のデータは挿入ソートで整列、
それらをマージしていく。

32個までは merge sort より insertion sort の方が速い

CPU キャッシュに当たりやすい順番で行う。

ティムソート (TIM SORT)

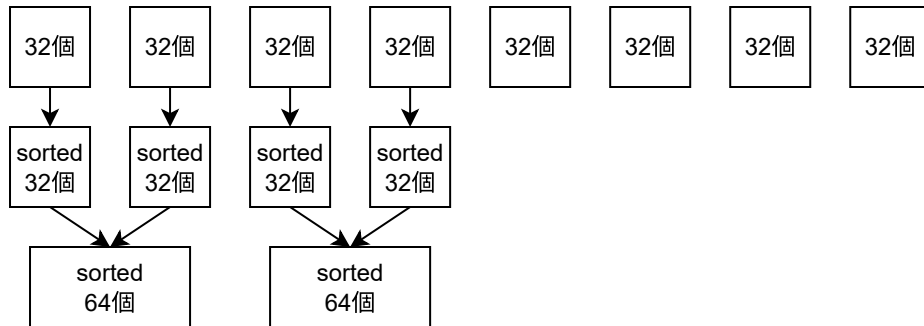


データを32個ずつに分割し、
32個のデータは挿入ソートで整列、
それらをマージしていく。

32個までは merge sortより insertion sortの方が速い

CPU キャッシュに当たりやすい順番で行う。

ティムソート (TIM SORT)

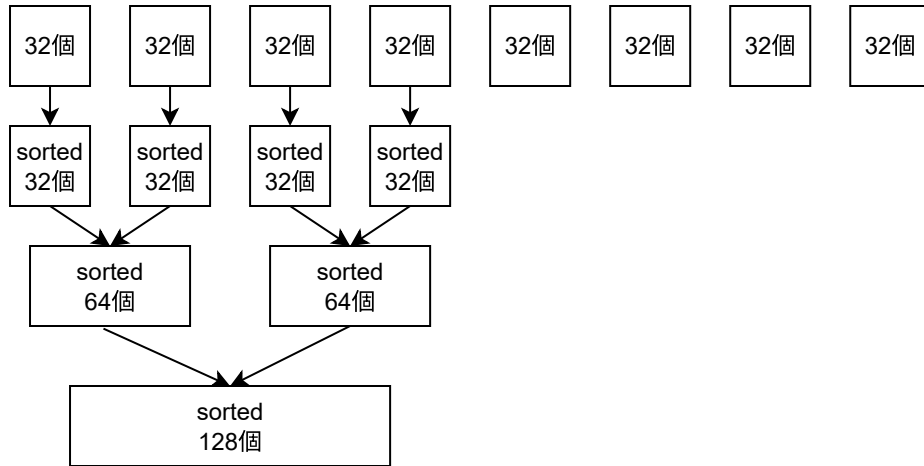


データを32個ずつに分割し、
32個のデータは挿入ソートで整列、
それらをマージしていく。

32個までは merge sort より insertion sort の方が速い

CPU キャッシュに当たりやすい順番で行う。

ティムソート (TIM SORT)

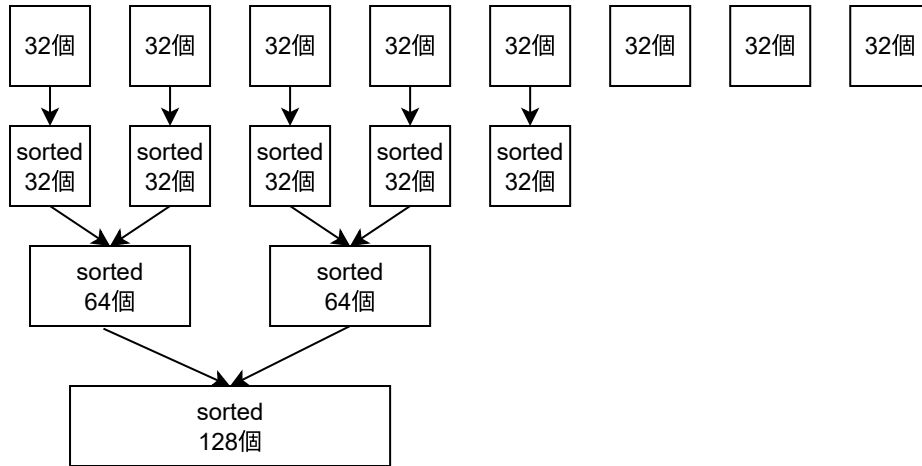


データを32個ずつに分割し、
32個のデータは挿入ソートで整列、
それらをマージしていく。

32個までは merge sort より insertion sort の方が速い

CPU キャッシュに当たりやすい順番で行う。

ティムソート (TIM SORT)

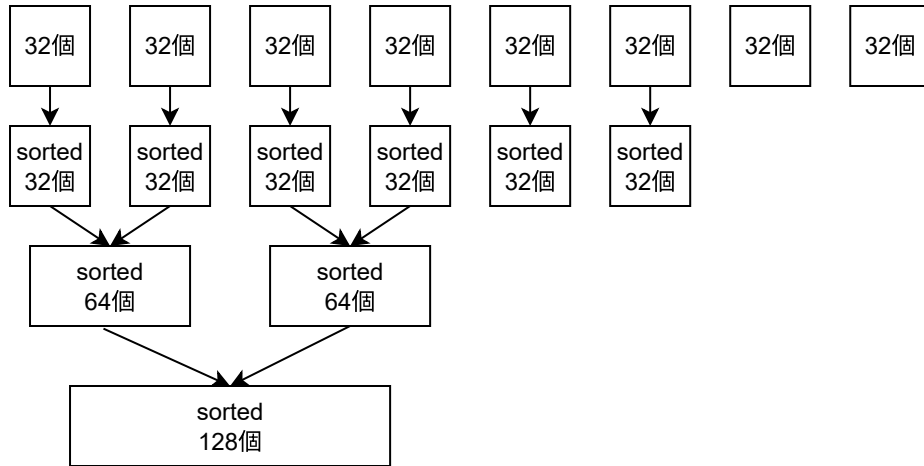


データを32個ずつに分割し、
32個のデータは挿入ソートで整列、
それらをマージしていく。

32個までは merge sort より insertion sort の方が速い

CPU キャッシュに当たりやすい順番で行う。

ティムソート (TIM SORT)

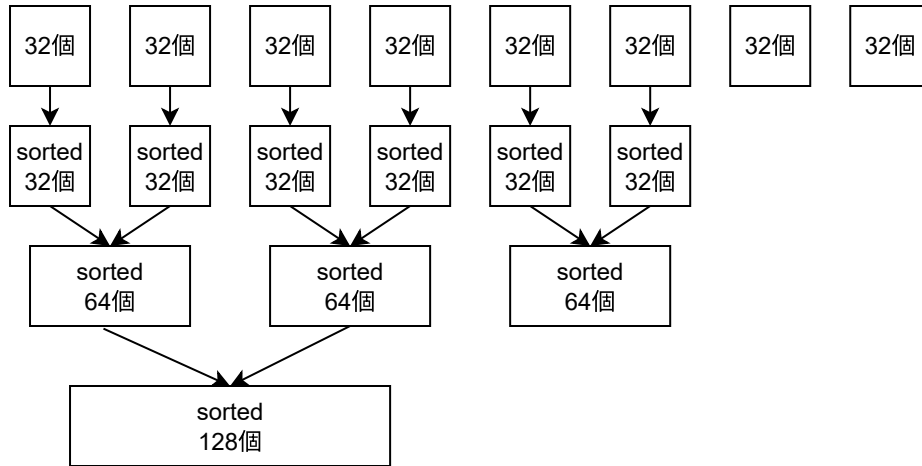


データを32個ずつに分割し、
32個のデータは挿入ソートで整列、
それらをマージしていく。

32個までは merge sortより insertion sortの方が速い

CPU キャッシュに当たりやすい順番で行う。

ティムソート (TIM SORT)

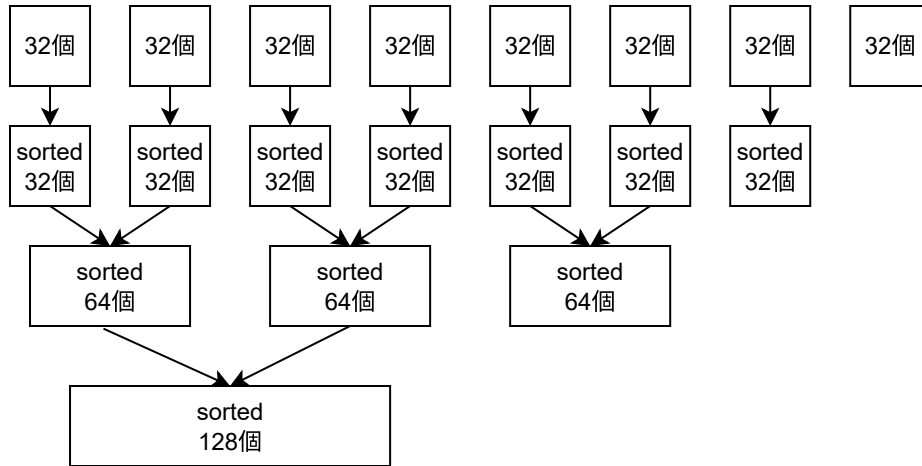


データを32個ずつに分割し、
32個のデータは挿入ソートで整列、
それらをマージしていく。

32個までは merge sortより insertion sortの方が速い

CPU キャッシュに当たりやすい順番で行う。

ティムソート (TIM SORT)

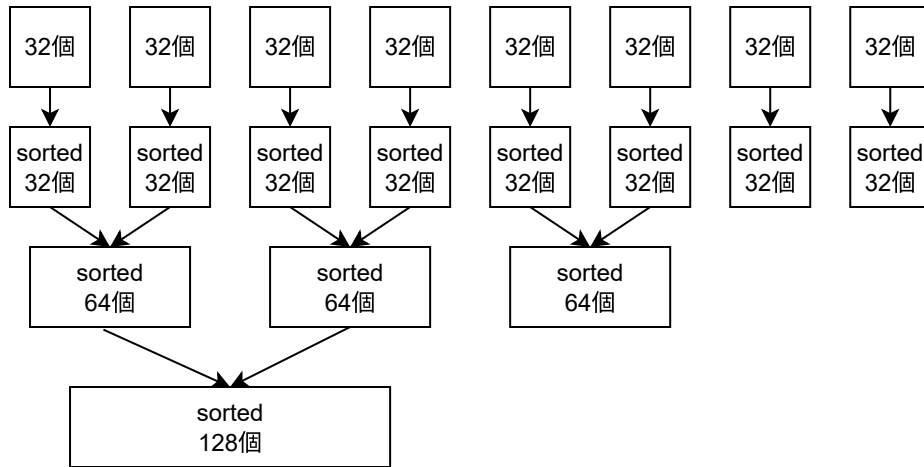


データを32個ずつに分割し、
32個のデータは挿入ソートで整列、
それらをマージしていく。

32個までは merge sortより insertion sortの方が速い

CPU キャッシュに当たりやすい順番で行う。

ティムソート (TIM SORT)

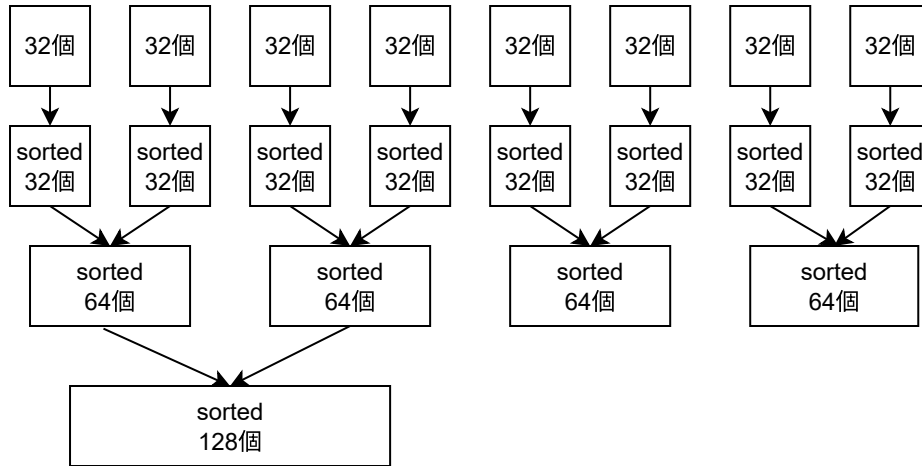


データを32個ずつに分割し、
32個のデータは挿入ソートで整列、
それらをマージしていく。

32個までは merge sortより insertion sortの方が速い

CPU キャッシュに当たりやすい順番で行う。

ティムソート (TIM SORT)

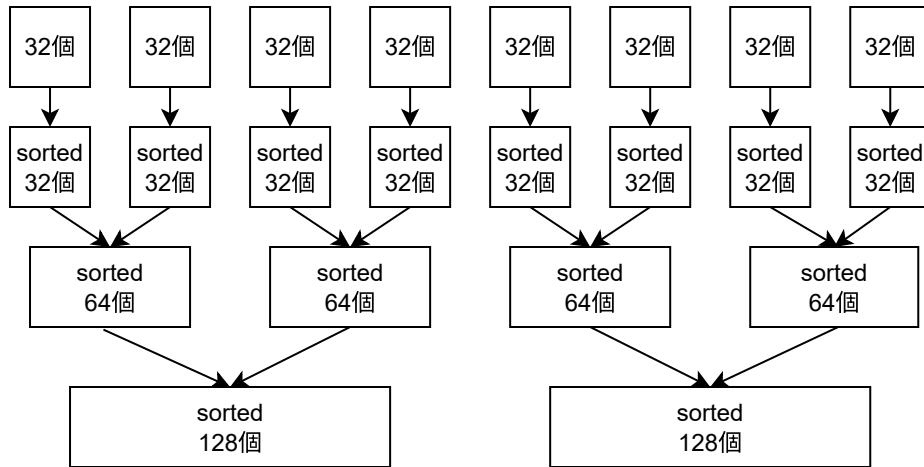


データを32個ずつに分割し、
32個のデータは挿入ソートで整列、
それらをマージしていく。

32個までは merge sortより insertion sortの方が速い

CPU キャッシュに当たりやすい順番で行う。

ティムソート (TIM SORT)

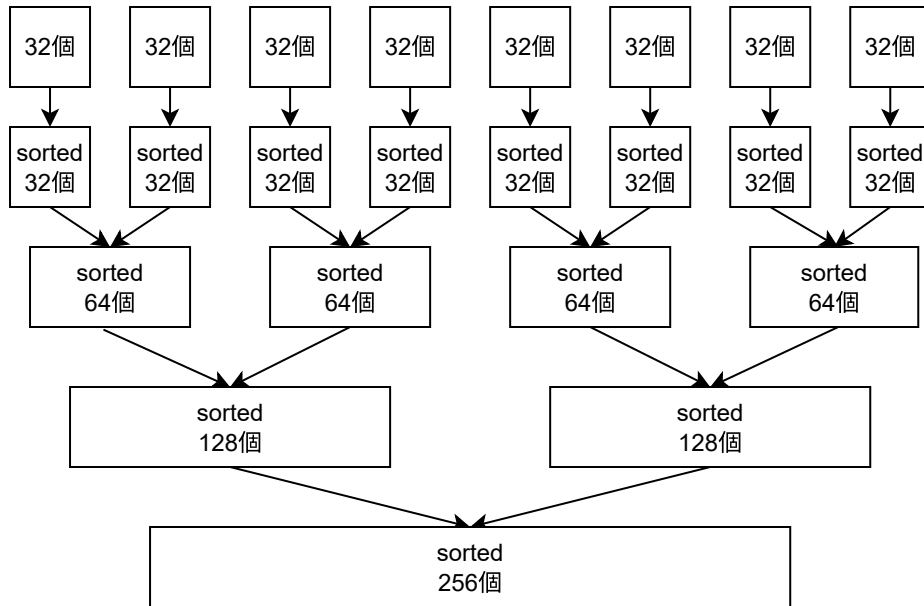


データを32個ずつに分割し、
32個のデータは挿入ソートで整列、
それらをマージしていく。

32個までは merge sortより insertion sortの方が速い

CPU キャッシュに当たりやすい順番で行う。

ティムソート (TIM SORT)



データを32個ずつに分割し、
32個のデータは挿入ソートで整列、
それらをマージしていく。

32個までは merge sort より insertion sort の方が速い

CPU キャッシュに当たりやすい順番で行う。

バケットソート・計数ソート・基数ソート

バケットソート (BUCKET SORT)

一定範囲の区分にデータを区分けして、各区分をソートし結合。

例: 100円以下の領収証・101-200円の領収証・201-300円の領収証...に分類後ソート
 n 個のデータが k 個バケツに均等に散らばれば早くソートできる $O(n + k)$ 。

計数ソート (COUNTING SORT)

種類ごとの数を数えて、ソート

例: 1円が5枚, 5円が3枚, 10円が12枚, ...

データの種類が少なければ早くソートできる $O(n + k)$ 。

基数ソート (RADIX SORT)

下1桁で安定ソート, 2桁目で安定ソート, ... を繰り返す。

桁数 $k \times$ データ数 n でソート可能 $O(kn)$ 。

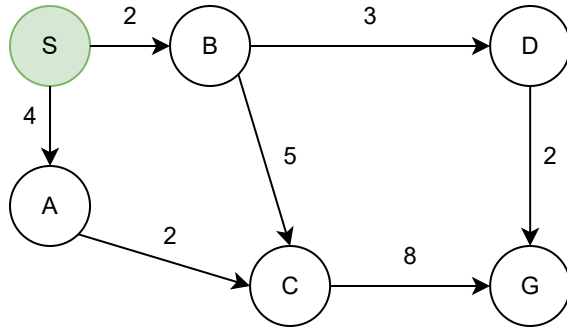
事前に分かっているデータの性質を使って高速にソート

グラフアルゴリズム - GRAPH ALGORITHM

最短経路 - ダイクストラ法(DIJKSTRA'S)

S から G までの最短距離を求めたい。

点を1つずつ、S からの最短距離が短い順番で、最短距離を確定させる。

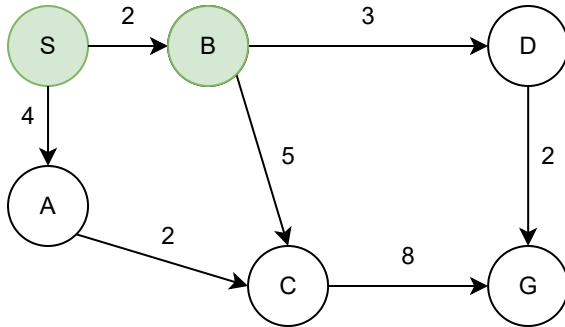


確定: S:0, 次に行ける点: B:2, A:4

最短経路 - ダイクストラ法(DIJKSTRA'S)

S から G までの最短距離を求めたい。

点を1つずつ、S からの最短距離が短い順番で、最短距離を確定させる。



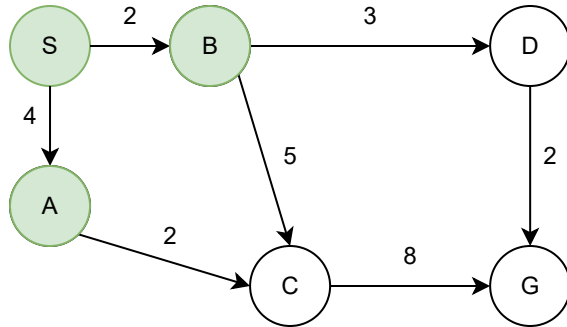
確定: S:0, 次に行ける点: B:2, A:4

確定: B:2, 次に行ける点 A:4, D:5, C:7

最短経路 - ダイクストラ法(DIJKSTRA'S)

S から G までの最短距離を求めたい。

点を1つずつ、S からの最短距離が短い順番で、最短距離を確定させる。



確定: S:0, 次に行ける点: B:2, A:4

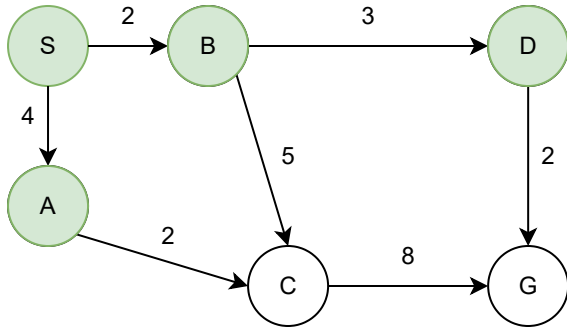
確定: B:2, 次に行ける点 A:4, D:5, C:7

確定: A:4, 次に行ける点 D:5, C:6

最短経路 - ダイクストラ法(DIJKSTRA'S)

S から G までの最短距離を求めたい。

点を1つずつ、S からの最短距離が短い順番で、最短距離を確定させる。



確定: S:0, 次に行ける点: B:2, A:4

確定: B:2, 次に行ける点 A:4, D:5, C:7

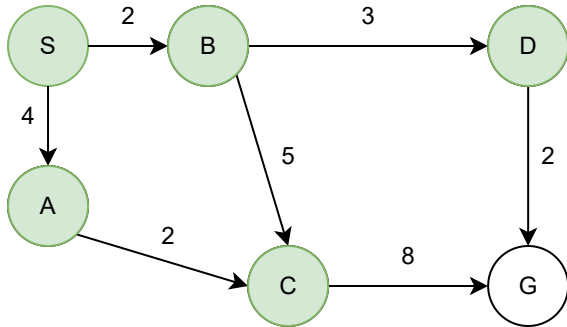
確定: A:4, 次に行ける点 D:5, C:6

確定: D:5, 次に行ける点 C:6, G:7

最短経路 - ダイクストラ法(DIJKSTRA'S)

S から G までの最短距離を求めたい。

点を1つずつ、S からの最短距離が短い順番で、最短距離を確定させる。



確定: S:0, 次に行ける点: B:2, A:4

確定: B:2, 次に行ける点 A:4, D:5, C:7

確定: A:4, 次に行ける点 D:5, C:6

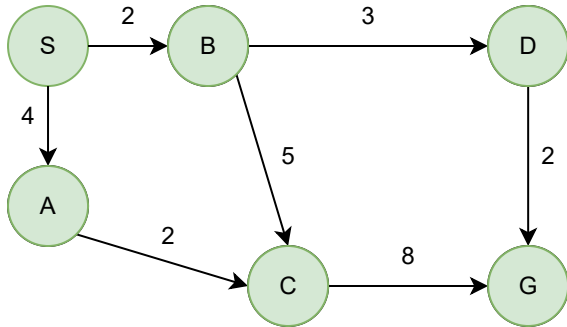
確定: D:5, 次に行ける点 C:6, G:7

確定: C:5, 次に行ける点 G:7

最短経路 - ダイクストラ法(DIJKSTRA'S)

S から G までの最短距離を求めたい。

点を1つずつ、S からの最短距離が短い順番で、最短距離を確定させる。



確定: S:0, 次に行ける点: B:2, A:4

確定: B:2, 次に行ける点 A:4, D:5, C:7

確定: A:4, 次に行ける点 D:5, C:6

確定: D:5, 次に行ける点 C:6, G:7

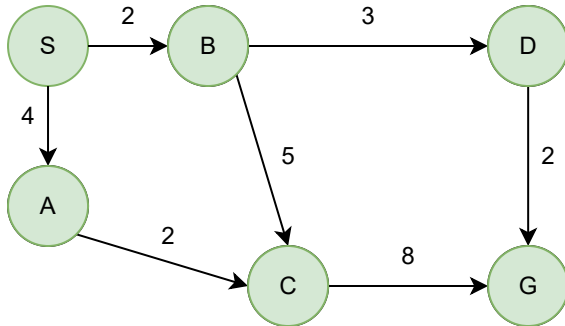
確定: C:5, 次に行ける点 G:7

確定: G:7

最短経路 - ダイクストラ法(DIJKSTRA'S)

S から G までの最短距離を求めたい。

点を1つずつ、Sからの最短距離が短い順番で、最短距離を確定させる。



確定: S:0, 次に行ける点: B:2, A:4

確定: B:2, 次に行ける点 A:4, D:5, C:7

確定: A:4, 次に行ける点 D:5, C:6

確定: D:5, 次に行ける点 C:6, G:7

確定: C:5, 次に行ける点 G:7

確定: G:7

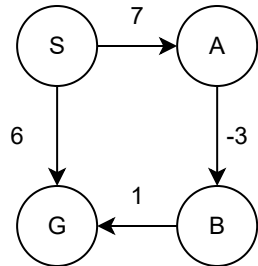
計算時間 $O((E + V)\log(V))$

負の枝があるとなぜ使えない(最短が順番に決まらない)

最短経路 - ワーシャルフロイド法 (FLOYD-WARSHALL'S)

S から G までの最短距離を求めたい。

全点(p)について、全二点間の距離テーブルを、pを経由したら近くならないか更新する



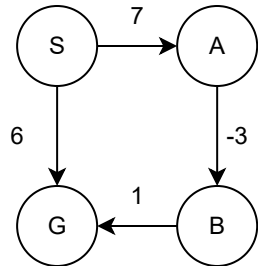
初期

$$\begin{pmatrix} 0 & 7 & \text{inf} & 6 \\ \text{inf} & 0 & -3 & \text{inf} \\ \text{inf} & \text{inf} & 0 & 1 \\ \text{inf} & \text{inf} & \text{inf} & 0 \end{pmatrix}$$

最短経路 - ワーシャルフロイド法 (FLOYD-WARSHALL'S)

S から G までの最短距離を求めたい。

全点(p)について、全二点間の距離テーブルを、pを経由したら近くならないか更新する



初期

$$\begin{pmatrix} 0 & 7 & \text{inf} & 6 \\ \text{inf} & 0 & -3 & \text{inf} \\ \text{inf} & \text{inf} & 0 & 1 \\ \text{inf} & \text{inf} & \text{inf} & 0 \end{pmatrix}$$

Sを経由(省略)

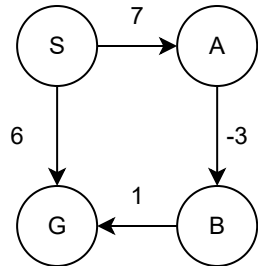
Aを経由

$$\begin{pmatrix} 0 & 7 & 4 & 6 \\ \text{inf} & 0 & -3 & \text{inf} \\ \text{inf} & \text{inf} & 0 & 1 \\ \text{inf} & \text{inf} & \text{inf} & 0 \end{pmatrix}$$

最短経路 - ワーシャルフロイド法 (FLOYD-WARSHALL'S)

S から G までの最短距離を求めたい。

全点(p)について、全二点間の距離テーブルを、pを経由したら近くならないか更新する



初期

$$\begin{pmatrix} 0 & 7 & \text{inf} & 6 \\ \text{inf} & 0 & -3 & \text{inf} \\ \text{inf} & \text{inf} & 0 & 1 \\ \text{inf} & \text{inf} & \text{inf} & 0 \end{pmatrix}$$

Sを経由(省略)

Aを経由

$$\begin{pmatrix} 0 & 7 & 4 & 6 \\ \text{inf} & 0 & -3 & \text{inf} \\ \text{inf} & \text{inf} & 0 & 1 \\ \text{inf} & \text{inf} & \text{inf} & 0 \end{pmatrix}$$

Bを経由

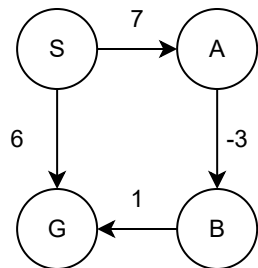
$$\begin{pmatrix} 0 & 7 & 4 & 5 \\ \text{inf} & 0 & -3 & -2 \\ \text{inf} & \text{inf} & 0 & 1 \\ \text{inf} & \text{inf} & \text{inf} & 0 \end{pmatrix}$$

Gを経由(省略)

最短経路 - ワーシャルフロイド法 (FLOYD-WARSHALL'S)

S から G までの最短距離を求めたい。

全点(p)について、全二点間の距離テーブルを、pを経由したら近くならないか更新する



初期

$$\begin{pmatrix} 0 & 7 & \text{inf} & 6 \\ \text{inf} & 0 & -3 & \text{inf} \\ \text{inf} & \text{inf} & 0 & 1 \\ \text{inf} & \text{inf} & \text{inf} & 0 \end{pmatrix}$$

Sを経由(省略)

Aを経由

$$\begin{pmatrix} 0 & 7 & 4 & 6 \\ \text{inf} & 0 & -3 & \text{inf} \\ \text{inf} & \text{inf} & 0 & 1 \\ \text{inf} & \text{inf} & \text{inf} & 0 \end{pmatrix}$$

Bを経由

$$\begin{pmatrix} 0 & 7 & 4 & 5 \\ \text{inf} & 0 & -3 & -2 \\ \text{inf} & \text{inf} & 0 & 1 \\ \text{inf} & \text{inf} & \text{inf} & 0 \end{pmatrix}$$

Gを経由(省略)

計算時間: $O(V^3)$

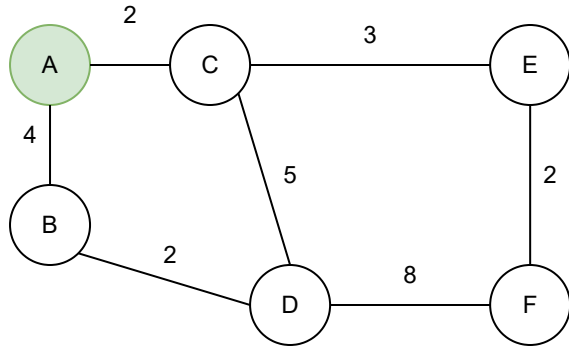
全点間の最短距離が同時に求まる。

負の辺が有っても計算可能。

最後の対角成分 (点から自分への点) にマイナスがあれば負のループが存在する

最小全域木 - プリム法 (PRIM'S)

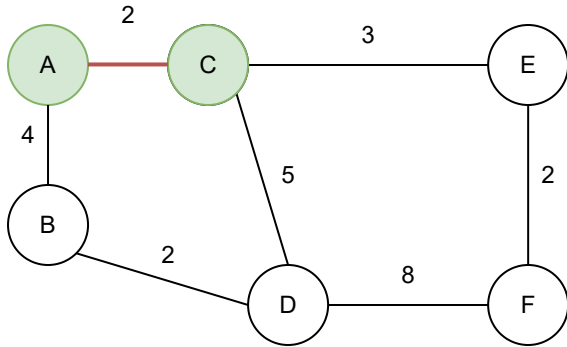
グラフから枝を $V-1$ 本選んで、最低コストで連結にしたい。
点 S を含む2点の最低コスト木, 3点の最低コスト木.. と木を成長させる。



確定: A:0, 次に行ける点: C:2, B:4

最小全域木 - プリム法 (PRIM'S)

グラフから枝を $V-1$ 本選んで、最低コストで連結にしたい。
点 S を含む2点の最低コスト木, 3点の最低コスト木.. と木を成長させる。

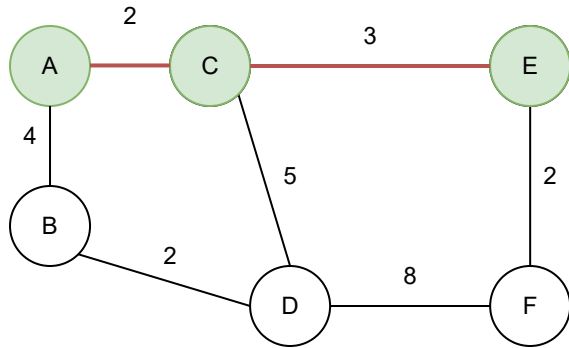


確定: A:0, 次に行ける点: C:2, B:4

確定: C(コスト+2), 次に行ける点 E:3, B:4, D:5

最小全域木 - プリム法 (PRIM'S)

グラフから枝を $V-1$ 本選んで、最低コストで連結にしたい。
点 S を含む2点の最低コスト木, 3点の最低コスト木.. と木を成長させる。



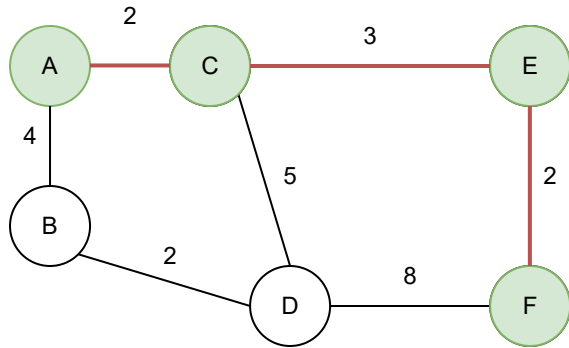
確定: A:0, 次に行ける点: C:2, B:4

確定: C(コスト+2), 次に行ける点 E:3, B:4, D:5

確定: E(コスト+3), 次に行ける点 F:2, B:4, D:5

最小全域木 - プリム法 (PRIM'S)

グラフから枝を $V-1$ 本選んで、最低コストで連結にしたい。
点 S を含む 2 点の最低コスト木, 3 点の最低コスト木.. と木を成長させる。



確定: A:0, 次に行ける点: C:2, B:4

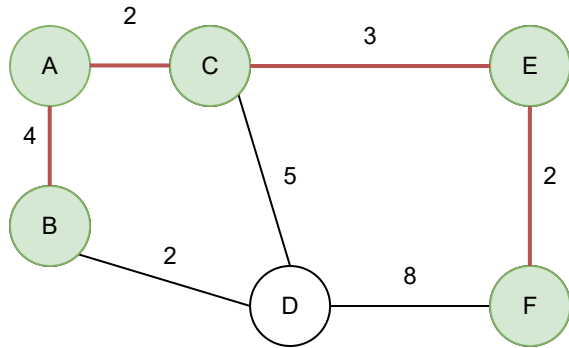
確定: C(コスト+2), 次に行ける点 E:3, B:4, D:5

確定: E(コスト+3), 次に行ける点 F:2, B:4, D:5

確定: F(コスト+2), 次に行ける点 B:4, D:5

最小全域木 - プリム法 (PRIM'S)

グラフから枝を $V-1$ 本選んで、最低コストで連結にしたい。
 点 S を含む 2 点の最低コスト木, 3 点の最低コスト木.. と木を成長させる。



確定: A:0, 次に行ける点: C:2, B:4

確定: C(コスト+2), 次に行ける点 E:3, B:4, D:5

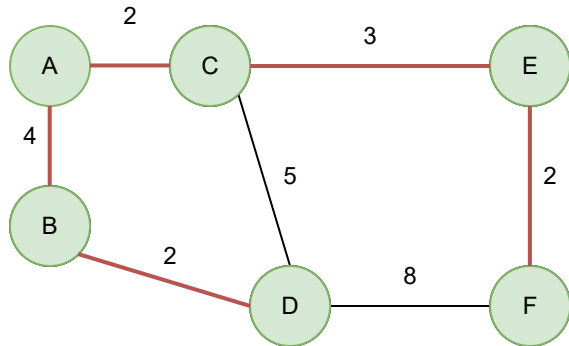
確定: E(コスト+3), 次に行ける点 F:2, B:4, D:5

確定: F(コスト+2), 次に行ける点 B:4, D:5

確定: B(コスト+4), 次に行ける点 D:2

最小全域木 - プリム法 (PRIM'S)

グラフから枝を $V-1$ 本選んで、最低コストで連結にしたい。
 点 S を含む 2 点の最低コスト木, 3 点の最低コスト木.. と木を成長させる。



確定: A:0, 次に行ける点: C:2, B:4

確定: C(コスト+2), 次に行ける点 E:3, B:4, D:5

確定: E(コスト+3), 次に行ける点 F:2, B:4, D:5

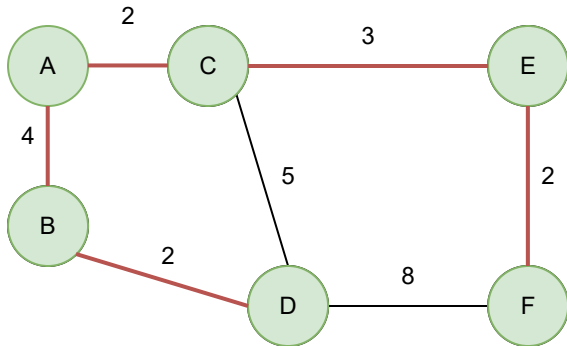
確定: F(コスト+2), 次に行ける点 B:4, D:5

確定: B(コスト+4), 次に行ける点 D:2

確定: D(コスト+2)

最小全域木 - プリム法 (PRIM'S)

グラフから枝を $V-1$ 本選んで、最低コストで連結にしたい。
 点 S を含む 2 点の最低コスト木, 3 点の最低コスト木.. と木を成長させる。



確定: A:0, 次に行ける点: C:2, B:4

確定: C(コスト+2), 次に行ける点 E:3, B:4, D:5

確定: E(コスト+3), 次に行ける点 F:2, B:4, D:5

確定: F(コスト+2), 次に行ける点 B:4, D:5

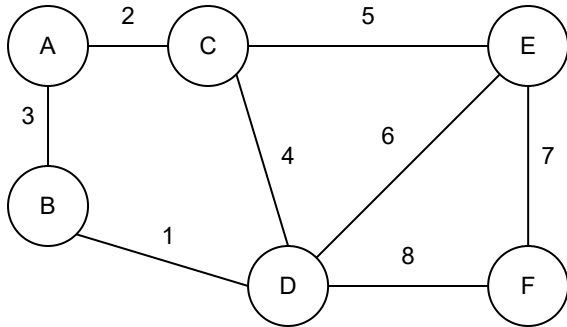
確定: B(コスト+4), 次に行ける点 D:2

確定: D(コスト+2)

計算時間 $O((E + V)\log(V))$

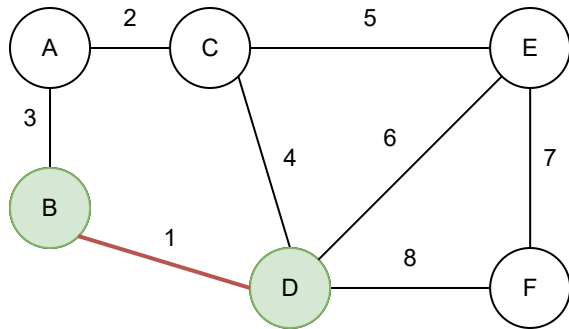
最小全域木 - クラスカル法 (KRUSKAL'S)

グラフから枝を $V-1$ 本選んで、最低コストで連結にしたい。
安い辺から順番に、採用が無意味でないものを採用していく



最小全域木 - クラスカル法 (KRUSKAL'S)

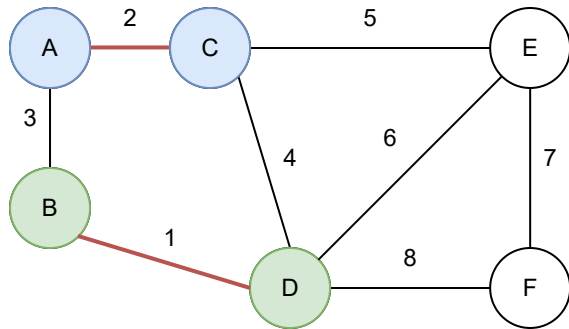
グラフから枝を $V-1$ 本選んで、最低コストで連結にしたい。
安い辺から順番に、採用が無意味でないものを採用していく



辺 B-D 採用 (コスト1)

最小全域木 - クラスカル法 (KRUSKAL'S)

グラフから枝を $V-1$ 本選んで、最低コストで連結にしたい。
安い辺から順番に、採用が無意味でないものを採用していく

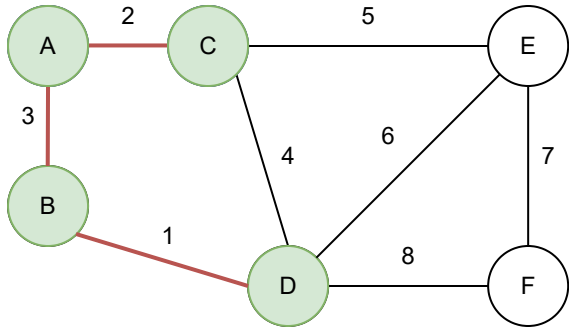


辺 B-D 採用 (コスト1)

辺 A-C 採用 (コスト2)

最小全域木 - クラスカル法 (KRUSKAL'S)

グラフから枝を $V-1$ 本選んで、最低コストで連結にしたい。
安い辺から順番に、採用が無意味でないものを採用していく



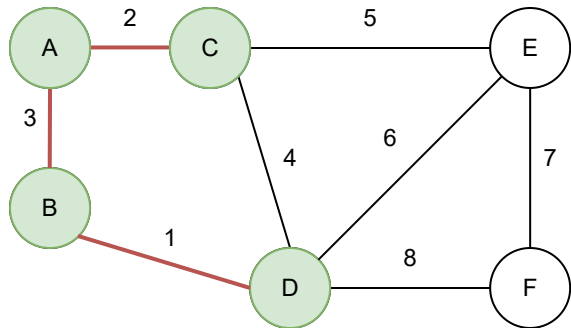
辺 B-D 採用 (コスト1)

辺 A-C 採用 (コスト2)

辺 A-B 採用 (コスト3)

最小全域木 - クラスカル法 (KRUSKAL'S)

グラフから枝を $V-1$ 本選んで、最低コストで連結にしたい。
安い辺から順番に、採用が無意味でないものを採用していく



辺 B-D 採用 (コスト1)

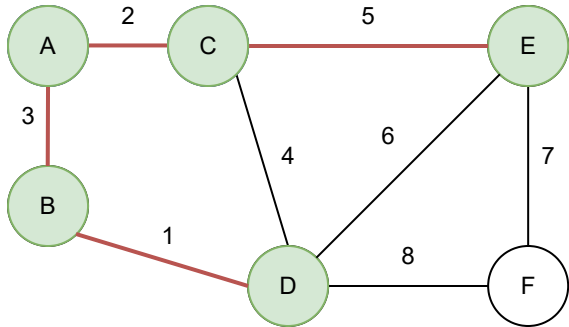
辺 A-C 採用 (コスト2)

辺 A-B 採用 (コスト3)

辺 C-D 不採用

最小全域木 - クラスカル法 (KRUSKAL'S)

グラフから枝を $V-1$ 本選んで、最低コストで連結にしたい。
安い辺から順番に、採用が無意味でないものを採用していく



辺 B-D 採用 (コスト1)

辺 A-C 採用 (コスト2)

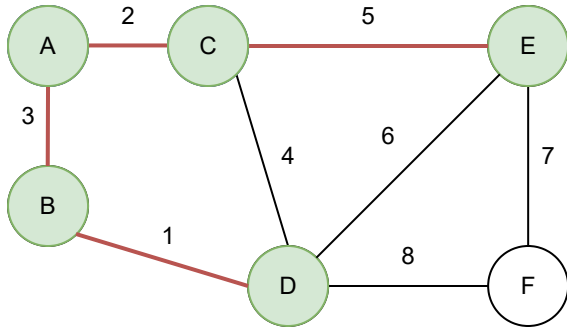
辺 A-B 採用 (コスト3)

辺 C-D 不採用

辺 C-E 採用 (コスト5)

最小全域木 - クラスカル法 (KRUSKAL'S)

グラフから枝を $V-1$ 本選んで、最低コストで連結にしたい。
安い辺から順番に、採用が無意味でないものを採用していく



辺 B-D 採用 (コスト1)

辺 A-C 採用 (コスト2)

辺 A-B 採用 (コスト3)

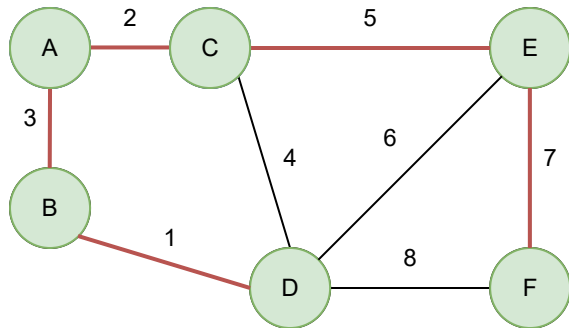
辺 C-D 不採用

辺 C-E 採用 (コスト5)

辺 D-E 採用

最小全域木 - クラスカル法 (KRUSKAL'S)

グラフから枝を $V-1$ 本選んで、最低コストで連結にしたい。
安い辺から順番に、採用が無意味でないものを採用していく



辺 B-D 採用 (コスト1)

辺 A-C 採用 (コスト2)

辺 A-B 採用 (コスト3)

辺 C-D 不採用

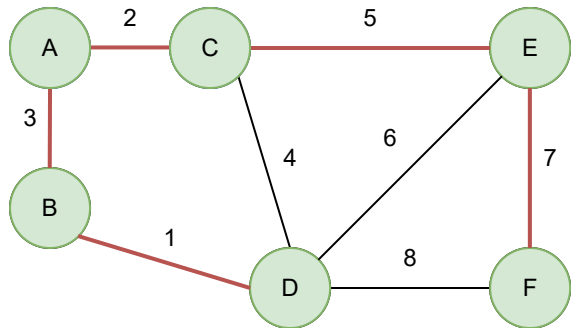
辺 C-E 採用 (コスト5)

辺 D-E 採用

辺 E-F 採用 (コスト7)

最小全域木 - クラスカル法 (KRUSKAL'S)

グラフから枝を $V-1$ 本選んで、最低コストで連結にしたい。
安い辺から順番に、採用が無意味でないものを採用していく



辺 B-D 採用 (コスト1)

辺 A-C 採用 (コスト2)

辺 A-B 採用 (コスト3)

辺 C-D 不採用

辺 C-E 採用 (コスト5)

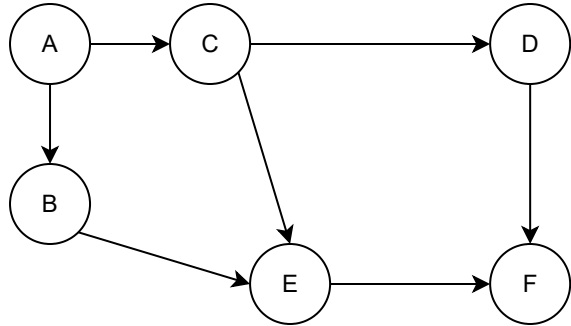
辺 D-E 採用

辺 E-F 採用 (コスト7)

計算時間 $O(E \log(E))$

トポロジカルソート (TOPOLOGICAL SORT)

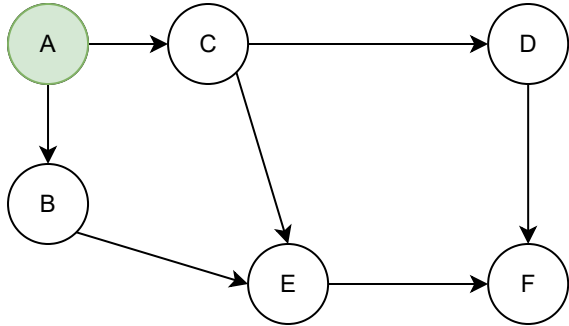
グラフの枝を、依存関係と見做して、実行可能な順番を求める。



Aは入次数が0なので、ここからスタート。

トポロジカルソート (TOPOLOGICAL SORT)

グラフの枝を、依存関係と見做して、実行可能な順番を求める。

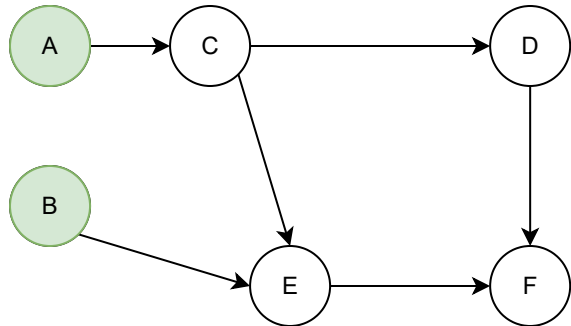


Aは入次数が0なので、ここからスタート。

A確定。A-B, A-Cを切断しに行く

トポロジカルソート (TOPOLOGICAL SORT)

グラフの枝を、依存関係と見做して、実行可能な順番を求める。



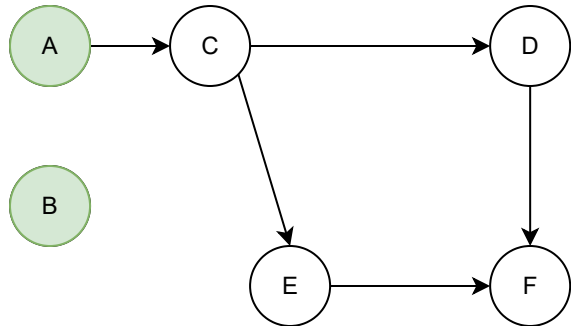
Aは入次数が0なので、ここからスタート。

A確定。A-B, A-Cを切断しに行く

A-Bを切断。B確定。B-Eが切断可能

トポロジカルソート (TOPOLOGICAL SORT)

グラフの枝を、依存関係と見做して、実行可能な順番を求める。



Aは入次数が0なので、ここからスタート。

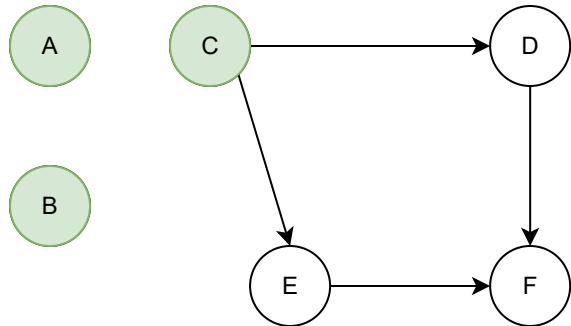
A確定。A-B, A-Cを切断しに行く

A-Bを切断。B確定。B-Eが切断可能

B-Eを切断。Eはまだ実行不可能。

トポロジカルソート (TOPOLOGICAL SORT)

グラフの枝を、依存関係と見做して、実行可能な順番を求める。



Aは入次数が0なので、ここからスタート。

A確定。A-B, A-Cを切断しに行く

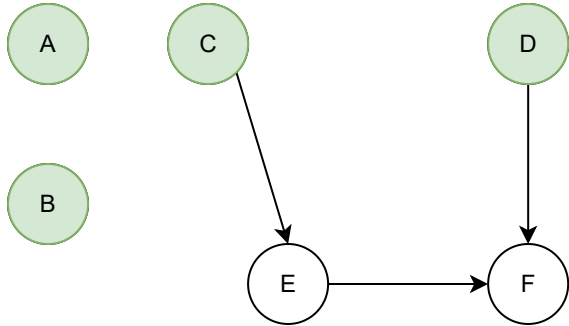
A-Bを切断。B確定。B-Eが切断可能

B-Eを切断。Eはまだ実行不可能。

A-Cを切断。C確定。C-D, C-Eが切断可能

トポロジカルソート (TOPOLOGICAL SORT)

グラフの枝を、依存関係と見做して、実行可能な順番を求める。



Aは入次数が0なので、ここからスタート。

A確定。A-B, A-Cを切断しに行く

A-Bを切断。B確定。B-Eが切断可能

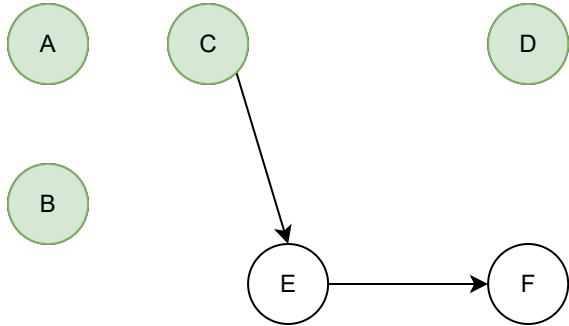
B-Eを切断。Eはまだ実行不可能。

A-Cを切断。C確定。C-D, C-Eが切断可能

C-Dを切断。D確定。D-Fが切断可能

トポロジカルソート (TOPOLOGICAL SORT)

グラフの枝を、依存関係と見做して、実行可能な順番を求める。



Aは入次数が0なので、ここからスタート。

A確定。A-B, A-Cを切断しに行く

A-Bを切断。B確定。B-Eが切断可能

B-Eを切断。Eはまだ実行不可能。

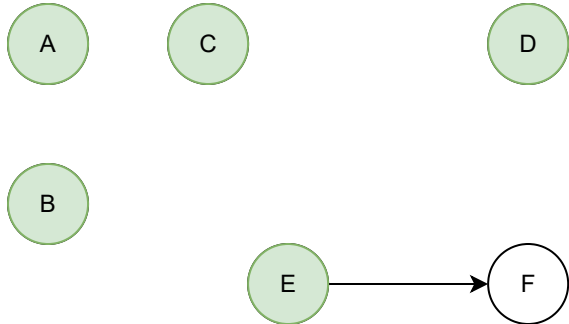
A-Cを切断。C確定。C-D, C-Eが切断可能

C-Dを切断。D確定。D-Fが切断可能

D-Fを切断。Fはまだ実行不可能。

トポロジカルソート (TOPOLOGICAL SORT)

グラフの枝を、依存関係と見做して、実行可能な順番を求める。



Aは入次数が0なので、ここからスタート。

A確定。A-B, A-Cを切断しに行く

A-Bを切断。B確定。B-Eが切断可能

B-Eを切断。Eはまだ実行不可能。

A-Cを切断。C確定。C-D, C-Eが切断可能

C-Dを切断。D確定。D-Fが切断可能

D-Fを切断。Fはまだ実行不可能。

C-Eを切断。E確定。E-Fが切断可能

トポロジカルソート (TOPOLOGICAL SORT)

グラフの枝を、依存関係と見做して、実行可能な順番を求める。



Aは入次数が0なので、ここからスタート。



A確定。A-B, A-Cを切断しに行く



A-Bを切断。B確定。B-Eが切断可能

B-Eを切断。Eはまだ実行不可能。

A-Cを切断。C確定。C-D, C-Eが切断可能

C-Dを切断。D確定。D-Fが切断可能

D-Fを切断。Fはまだ実行不可能。

C-Eを切断。E確定。E-Fが切断可能

E-Fを切断。F確定。全部実行できた。

トポロジカルソート (TOPOLOGICAL SORT)

グラフの枝を、依存関係と見做して、実行可能な順番を求める。



Aは入次数が0なので、ここからスタート。



A確定。A-B, A-Cを切断しに行く



A-Bを切断。B確定。B-Eが切断可能

B-Eを切断。Eはまだ実行不可能。

A-Cを切断。C確定。C-D, C-Eが切断可能

C-Dを切断。D確定。D-Fが切断可能

D-Fを切断。Fはまだ実行不可能。

C-Eを切断。E確定。E-Fが切断可能

E-Fを切断。F確定。全部実行できた。

計算時間 $O(E + V)$

マッチング - 文字列アルゴリズム

文字列探索

文字列Sに文字列Mが含まれている場所を探したい。

「うどんよりそば」に「どんより」は含まれる(2文字目から)

「アアアアアアアアアアアアアア」に「アアアアララ」は含まれるか?

ア	ア	ア	ア	ア	ア	ア	ア	ア	ア	ア	ア	ア	ア	ア
ア	ア	ア	ア	ラ	ラ									
	ア	ア	ア	ア	ラ	ラ								
		ア	ア	ア	ア	ラ	ラ							
			ア	ア	ア	ア	ラ	ラ						
				ア	ア	ア	ア	ラ	ラ					

...

愚直に探すと非常に遅くなる $O(SM)$ ケースがある。

効率的なアルゴリズムを考えたい。

ボイヤ・ムーア法 (BM法 / BOYER-MOORE)

P の後ろの文字から一致をテストする。

- マッチしなかったSの文字が次にPに含まれる場所
- マッチした部分がPに重なることのできる次の場所

の有利な方にずらしていく。

ア	ブ	ラ	カ	タ	ブ	ラ	タ	ア	ラ	ブ	ラ	ブ	ラ	ブ	ラ	カ	タ	ブ	ラ
ア	ア	ブ	ラ	ラ	ア	ブ	ラ												

1. まずはPを先頭に置く。最後の文字がマッチしない(相手は「タ」)
「タ」はPに無いので、「タ」の次までずらす。

ボイヤ・ムーア法 (BM法 / BOYER-MOORE)

P の後ろの文字から一致をテストする。

- マッチしなかったSの文字が次にPに含まれる場所
- マッチした部分がPに重なることのできる次の場所

の有利な方にずらしていく。

ア	ブ	ラ	カ	タ	ブ	ラ	タ	ア	ラ	ブ	ラ	ブ	ラ	ブ	ラ	カ	タ	ブ	ラ
ア	ア	ブ	ラ	ラ	ア	ブ	ラ												
							ア	ア	ブ	ラ	ラ	ア	ブ	ラ					

1. まずはPを先頭に置く。最後の文字がマッチしない(相手は「タ」)
「タ」はPに無いので、「タ」の次までずらす。
2. 後ろから3文字見て、相手の「ラ」がマッチしない。
マッチした「ブラ」に着目すると、4文字ずれすことができる。

ボイヤ・ムーア法 (BM法 / BOYER-MOORE)

P の後ろの文字から一致をテストする。

- マッチしなかったSの文字が次にPに含まれる場所
- マッチした部分がPに重なることのできる次の場所

の有利な方にずらしていく。

ア	ブ	ラ	カ	タ	ブ	ラ	タ	ア	ラ	ブ	ラ	ブ	ラ	ブ	ラ	カ	タ	ブ	ラ		
ア	ア	ブ	ラ	ラ	ア	ブ	ラ														
								ア	ア	ブ	ラ	ラ	ア	ブ	ラ						
														ア	ア	ブ	ラ	ラ	ア	ブ	ラ

1. まずはPを先頭に置く。最後の文字がマッチしない(相手は「タ」)
「タ」はPに無いので、「タ」の次までずらす。
2. 後ろから3文字見て、相手の「ラ」がマッチしない。
マッチした「ブラ」に着目すると、4文字ずれすことができる。
3. 以下続く

ボイヤ・ムーア法 (BM法 / BOYER-MOORE)

P の後ろの文字から一致をテストする。

- マッチしなかったSの文字が次にPに含まれる場所
- マッチした部分がPに重なることのできる次の場所

の有利な方にずらしていく。

ア	ブ	ラ	カ	タ	ブ	ラ	タ	ア	ラ	ブ	ラ	ブ	ラ	ブ	ラ	カ	タ	ブ	ラ
ア	ア	ブ	ラ	ラ	ア	ブ	ラ												
								ア	ア	ブ	ラ	ラ	ア	ブ	ラ				
												ア	ア	ブ	ラ	ラ	ア	ブ	ラ

1. まずはPを先頭に置く。最後の文字がマッチしない(相手は「タ」)
「タ」はPに無いので、「タ」の次までずらす。
2. 後ろから3文字見て、相手の「ラ」がマッチしない。
マッチした「ブラ」に着目すると、4文字ずれすことができる。
3. 以下続く

ミスの文字が? なら? だけずらす。
最後?文字マッチ後のミスは? だけずらす
の2種類の表はを $O(P)$ で準備可能

最悪は $O(S \times P)$ だが、最良時は $O(S/P)$
実用上はSがどんどんスキップされ高速。
改良すると最悪 $O(S + P)$ に出来る (ガリル規則)

KMP法 (クヌース・モリス・プラット法)

先頭から見ていき、ミス発生時に後ろに巻き戻さずに進めていく。

ラ	ブ	ラ	ブ	ラ	ブ	ブ	ラ	ブ	ラ	ラ	ラ	ラ
ラ	ブ	ラ	ブ	ラ	ラ							

1. 先頭5文字マッチ。6文字目がミス。
 マッチ5文字の先頭/最後の3文字が一致。
 そこまでずらす。

KMP法 (クヌース・モリス・プラット法)

先頭から見ていき、ミス発生時に後ろに巻き戻さずに進めていく。

ラ	ブ	ラ	ブ	ラ	ブ	ブ	ラ	ブ	ラ	ラ	ラ	ラ
ラ	ブ	ラ	ブ	ラ	ラ							
		ラ	ブ	ラ	ブ	ラ	ラ					

1. 先頭5文字マッチ。6文字目がミス。
マッチ5文字の先頭/最後の3文字が一致。
そこまですらす。
2. 3文字一致なので、4文字目から探索。
4文字目はマッチして、5文字目がミス。
マッチ4文字の先頭と最後の2文字は一致。

KMP法 (クヌース・モリス・プラット法)

先頭から見ていき、ミス発生時に後ろに巻き戻さずに進めていく。

ラ	ブ	ラ	ブ	ラ	ブ	ブ	ラ	ブ	ラ	ラ	ラ	ラ
ラ	ブ	ラ	ブ	ラ	ラ							
		ラ	ブ	ラ	ブ	ラ	ラ					
				ラ	ブ	ラ	ブ	ラ	ラ			

1. 先頭5文字マッチ。6文字目がミス。
マッチ5文字の先頭/最後の3文字が一致。
そこまですらす。
2. 3文字一致なので、4文字目から探索。
4文字目はマッチして、5文字目がミス。
マッチ4文字の先頭と最後の2文字は一致。
3. 3文字目はマッチせず。
マッチの先頭と最後のは一致無し。
先頭から探索。

KMP法 (クヌース・モリス・プラット法)

先頭から見ていき、ミス発生時に後ろに巻き戻さずに進めていく。

ラ	ブ	ラ	ブ	ラ	ブ	ブ	ラ	ブ	ラ	ラ	ラ	ラ
ラ	ブ	ラ	ブ	ラ	ラ							
		ラ	ブ	ラ	ブ	ラ	ラ					
				ラ	ブ	ラ	ブ	ラ	ラ			
						ラ	ブ	ラ	ブ	ラ	ラ	

1. 先頭5文字マッチ。6文字目がミス。
マッチ5文字の先頭/最後の3文字が一致。
そこまですらす。
2. 3文字一致なので、4文字目から探索。
4文字目はマッチして、5文字目がミス。
マッチ4文字の先頭と最後の2文字は一致。
3. 3文字目はマッチせず。
マッチの先頭と最後のは一致無し。
先頭から探索。
4. 先頭がマッチしない。次へ。

KMP法 (クヌース・モリス・プラット法)

先頭から見ていき、ミス発生時に後ろに巻き戻さずに進めていく。

ラ	ブ	ラ	ブ	ラ	ブ	ブ	ラ	ブ	ラ	ラ	ラ	ラ
ラ	ブ	ラ	ブ	ラ	ラ							
		ラ	ブ	ラ	ブ	ラ	ラ					
				ラ	ブ	ラ	ブ	ラ	ラ			
						ラ	ブ	ラ	ブ	ラ	ラ	
								ラ	ブ	ラ	ブ	ラ
										ラ	ブ	ラ

1. 先頭5文字マッチ。6文字目がミス。
マッチ5文字の先頭/最後の3文字が一致。
そこまですらす。
2. 3文字一致なので、4文字目から探索。
4文字目はマッチして、5文字目がミス。
マッチ4文字の先頭と最後の2文字は一致。
3. 3文字目はマッチせず。
マッチの先頭と最後のは一致無し。
先頭から探索。
4. 先頭がマッチしない。次へ。
5. 以下続く。

M の先頭 X 文字がマッチした際、 マッチの
先頭最後からの V 文字が一致する。
というテーブルが $O(P)$ で準備できる。
マッチ自体の計算量は $O(S)$
全体として、 $O(S + P)$

最長共通部分文字列 (LCS: LONGEST COMMON SUBSEQUENCE)

二つの文字列の共通の部分で最長のものを求めたい。
「あんぱん」と「あぱまん」と、「あぱん」

	あ	ん	ぱ	ん
あ	1	1	1	1
ぱ	1	1	2	2
ま	1	1	2	2
ん	1	2	2	3

最長共通部分文字列 (LCS: LONGEST COMMON SUBSEQUENCE)

二つの文字列の共通の部分で最長のものを求めたい。
「あんぱん」と「あぱまん」と、「あぱん」

	あ	ん	ぱ	ん
あ	1	1	1	1
ぱ	1	1	2	2
ま	1	1	2	2
ん	1	2	2	3

時間計算量は $O(NM)$

正規表現マッチ (REGULAR EXPRESSION MATCH)

複数の文字列にマッチするパターン。

例: "10*(円|ドル)"

1の後に、0が何回か繰り返しても良く、その後に円かドルが続く文字列

正規表現マッチ (REGULAR EXPRESSION MATCH)

複数の文字列にマッチするパターン。

例: "10*(円|ドル)"

1の後に、0が何回か繰り返しても良く、その後に円かドルが続く文字列

1円, 1ドル, 10円, 10ドル, 1000000000000円, etc....

正規表現マッチ (REGULAR EXPRESSION MATCH)

複数の文字列にマッチするパターン。

例: "10*(円|ドル)"

1の後に、0が何回か繰り返しても良く、その後に円かドルが続く文字列

1円, 1ドル, 10円, 10ドル, 1000000000000円, etc....

以下のようなスゴロクを作ればよい

正規表現マッチ (REGULAR EXPRESSION MATCH)

複数の文字列にマッチするパターン。

例: "10*(円|ドル)"

1の後に、0が何回か繰り返しても良く、その後に円かドルが続く文字列

1円, 1ドル, 10円, 10ドル, 1000000000000円, etc....

以下のようなスゴロクを作ればよい

